

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Design and simulation of a 4DOF robot arm

PROJECT WORK IN INDUSTRIAL ROBOTICS

MASTER OF SCIENCE IN MECHANICAL ENGINEERING – MECHATRONICS AND ROBOTICS

Authors: Simone Bucci - 10769179
Thomas Monaco - 10697002
Erwin Rafael Novicio - 10782977

Advisor: Giberti Hermes
Academic Year: 2025-26

Contents

Contents ..... **III**

Introduction **1**

1 Kinematics Analysis **3**

1.1. Inverse Kinematics 3

1.2. Direct Kinematic and Working Space 4

2 Motion Planning **6**

2.1. Trajectory n.1: Lines and Parabolas method 6

2.2. Trajectory n. 2: Cubic Spline Method 7

2.3. Convalidation through Direct Kinematic 9

3 Dynamics analysis **11**

3.1. Torques with MATLAB 11

3.1.1. Jacobian matrices J - numerical 12

3.1.2. Time derivative of the Jacobian matrices $\dot{J}$ - numerical 12

3.1.3. Results 14

3.2. SIMSCAPE Model 15

3.3. Different Payload 16

4 Motor and transmission sizing **19**

4.1. Motor sizing for standard payload 20

4.2. Sizing with different payload 23

5 Possible Improvement **25**

6 Quality of the robot **27**

6.1. Ellipsoids of force 29

6.2. Ellipsoids of velocity 31

7 Robot model – CAD **33**

7.1. Assembly 33

7.2. Bearing Configurations 37

7.3. Pulleys 38

7.4. Transmission ratios 39

7.5. Solution to avoid thread ABS 41

8 EPSON RC+ 8.0 virtual simulation 42
8.1. Robot programming 42
8.2. Trajectory motion simulation 46
8.2.1. Trajectory 1 simulation 47
8.2.2. Trajectory 2 simulation 47
8.3. TCP: Position, Velocity and Acceleration 48
8.3.1. Position – Trajectory 1 and Trajectory 2 48
8.3.2. Speed – Trajectory 1 and Trajectory 2 50
8.3.3. Acceleration -Trajectory 1 and Trajectory 2 51
8.4. Joint: Position, Velocity and Acceleration 52
8.4.1. Joint Position – Trajectory 1 and Trajectory 2 53
8.4.2. Joint Velocity – Trajectory 1 and Trajectory 2 54
8.4.3. Joint Acceleration – Trajectory 1 and Trajectory 2 55

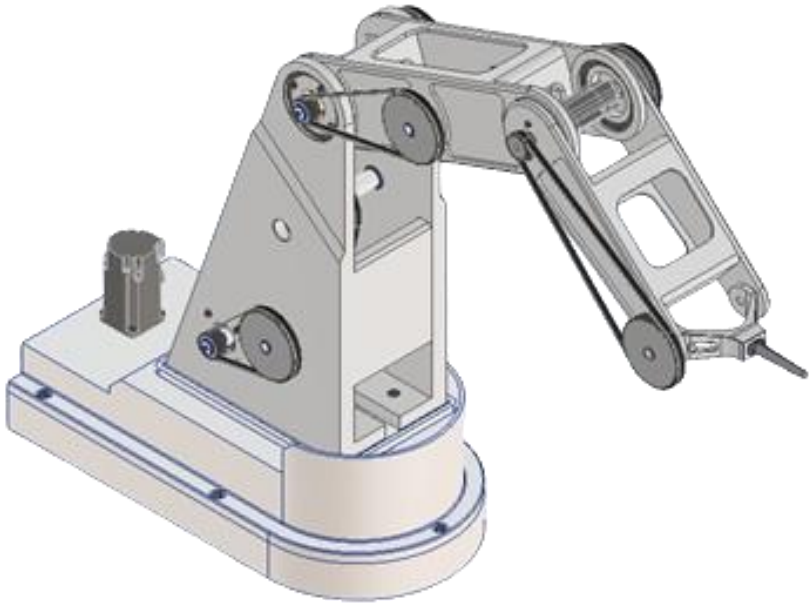
Introduction

This report is aimed to describe the workflow followed to design a 4DOF robot, from the definition of the workspace and trajectories to the 3D models, that has the task to move along two defined trajectories to perform different operation on a motorcycle helmet.

- **Specifications:**

In the following table are reported some of the geometrical and angular specifications of the designed robot (the numeration starts from the ground going to the EE):

Robot specifications	
Reach [mm]	
870	
Link lengths [mm]	
Link 1 (L1)	500
Link 2 (L2)	400
Link 3 (L3)	300
Link 4 (L4)	170
Joint range [deg]	
Joint 1	(-40°,40°)
Joint 2	(-52°,232°)
Joint 3	(-112°,112°)
Joint 4	(-120°,120°)
Link masses [kg]	
Link 1	13.74
Link 2	4.51
Link 3	2.18
Link 4	0.43



- **Trajectories**

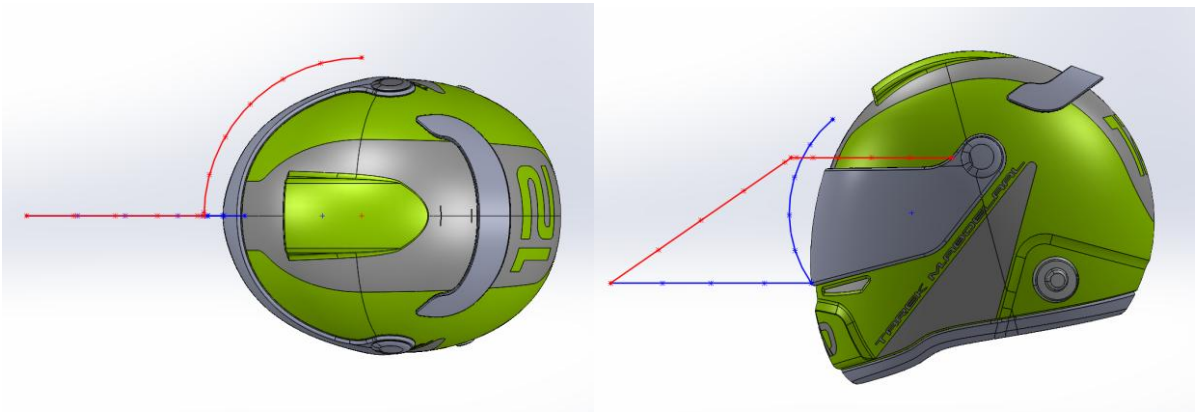


Figure 1: Trajectory 1 (blue) Trajectory 2 (red)

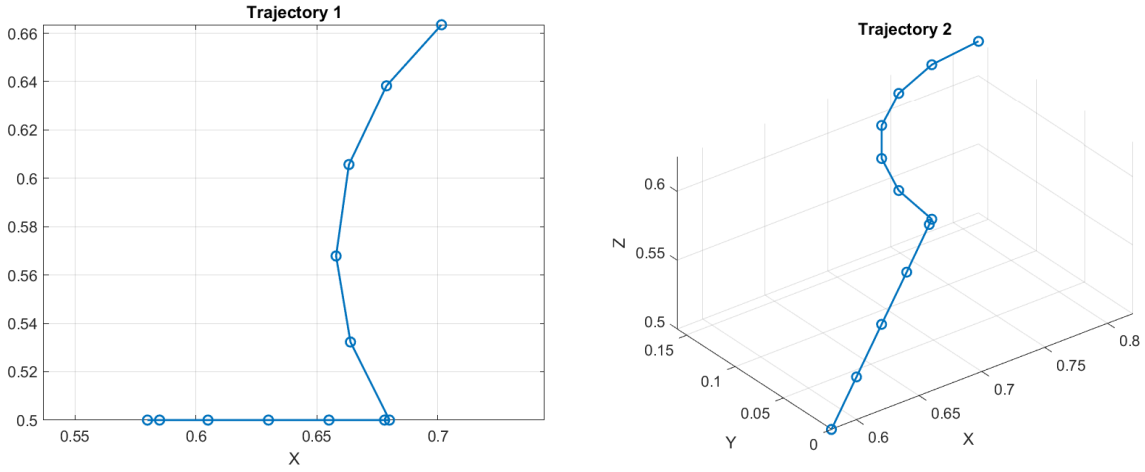


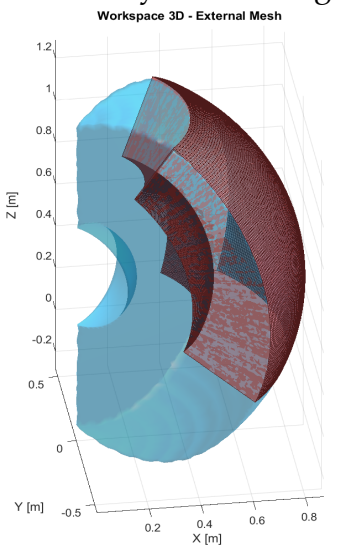
Figure 2 Points considered for both trajectories

- **Workspace:**

The blue region represents the theoretical workspace obtained by considering only the joint angular limits and the geometrical constraints of the robot.

The red region, instead, represents the ***effective*** workspace.

The more detailed analysis about how it is obtained will be handled in the following chapter.



1 Kinematics Analysis

To calculate the position of the end effector or any point belonging to the robot, the relationship between the working space and joint space variables must be determined. The workspace is the set S of the cartesian coordinates of the robot, and it consists of the cartesian coordinates of the end effector: x, y and z. The joint space is the set Q of the rotation each joint, q1, q2, q3 and q4, which correspond respectively to the rotation of the first, second, third and fourth joint.

The kinematics of a robot can be determined in two ways: Direct or Inverse kinematics.

1.1. Inverse Kinematics

In the Inverse kinematics, the working space variables is mapped into the joint space variables:

$$Q = F^{-1}(S) \rightarrow \begin{bmatrix} q_2 \\ q_3 \\ q_4 \end{bmatrix} = \begin{bmatrix} F_2^{-1}(s_1, s_2, s_3) \\ F_2^{-1}(s_1, s_2, s_3) \\ F_3^{-1}(s_1, s_2, s_3) \end{bmatrix}$$

It can be observed that q_1 isn't present on the system since it is independent with respect to the rotations q_2, q_3 and q_4.

For the sake of future calculations, these following variables were defined:

$$X = s_1 - L_4 \cos(\Phi) \cos(q_1)$$
$$Y = s_2 - L_4 \cos(\Phi) \sin(q_1)$$
$$Z = s_3 - L_4 \sin(\Phi)$$

Through trigonometry, the joint space variables can be calculated analytically as:

$$q_1 = \theta_1 = \tan^{-1}\left(\frac{s_2}{s_1}\right)$$

$$q_2 = \tan^{-1}\left(\frac{s_3}{\sqrt{s_1^2 + s_2^2}}\right) - \tan^{-1}\left(\frac{L_3 \sin(q_3) + L_4 \sin(q_3 + q_4)}{L_2 + L_3 \cos(q_3) + L_4 \cos(q_3 + q_4)}\right)$$

$$q_3 = \cos^{-1}\left(\frac{(X^2 + Y^2 + Z^2 - L_2^2 + L_3^2)}{2L_2L_3}\right)$$

$$q_4 = \Phi - q_2 - q_3$$

So basically, give in input the points of the trajectories we can find the configuration of angles (for the four joints).

It's important to notice that for a point in the space we will find two different solutions elbow up and elbow down. We consider the configuration elbow down.

1.2. Direct Kinematic and Working Space

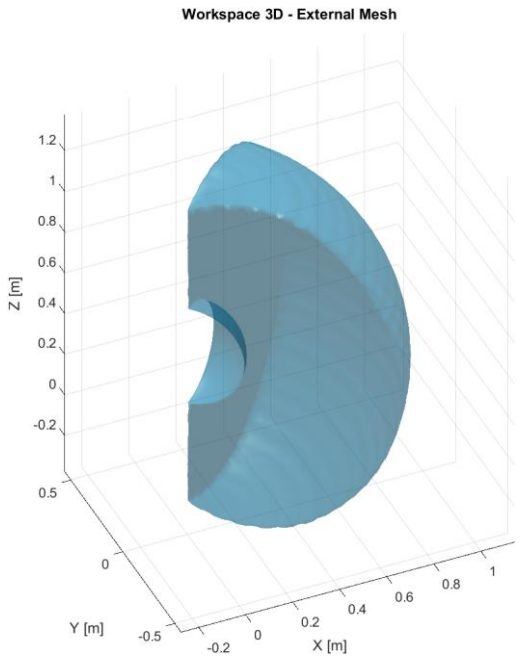
The forward (direct) kinematics provides the geometric mapping between the joint configuration of the manipulator and the position of its end–effector in the cartesian space.

$$S = F(Q).$$

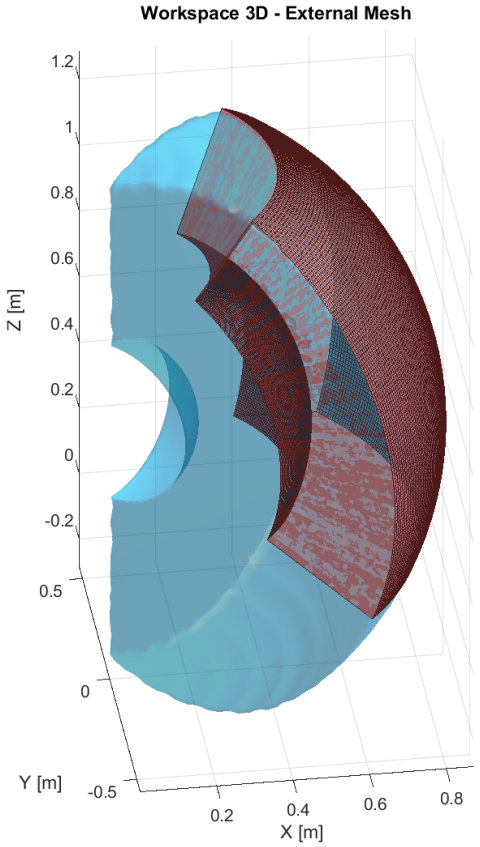
The analytical expression for the end-effector position is:

$$\begin{aligned} s_1 &= (L_2\cos q_2 + L_3\cos(q_2+q_3) + L_4\cos(q_2+q_3+q_4))\cos q_1, \\ s_2 &= (L_2\cos q_2 + L_3\cos(q_2+q_3) + L_4\cos(q_2+q_3+q_4))\sin q_1, \\ s_3 &= L_1 + L_2\sin q_2 + L_3\sin(q_2+q_3) + L_4\sin(q_2+q_3+q_4). \end{aligned}$$

The workspace is mapped thanks to the direct kinematics by creating a discrete grid in the joint space for q_2, q_3, q_4 and sampling multiple values of q_1 to exploit the rotational symmetry (take in account the physical limits). At each grid point, the direct kinematics function $\mathbf{S} = F(\mathbf{Q})$ computes the corresponding end-effector position. This procedure generates a set of points covering the reachable workspace. We at the end define a mesh to approximate the boundary points and found the limit of workspace.



Then we define the ***effective*** workspace (the red one): it is obtained by including only the reachable configurations of the type shown in Figure 3 and excluding the configurations that lead to poses of the type illustrated in Figure 4.



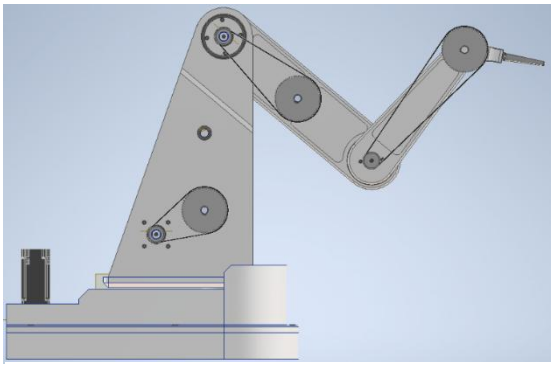


Figure 3 Desired operating configuration

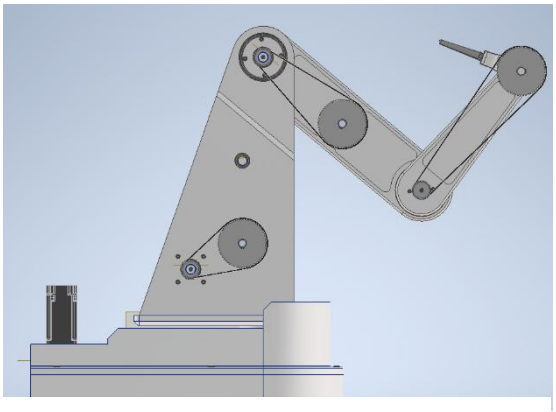


Figure 4 Operating configuration to avoid

To check if we perform a correct mapping of the workspace, we also use the Monte Carlo method, which is a statistical sampling technique. Instead of scanning the joint space on a structured grid, the joint variables are randomly sampled within their admissible ranges.

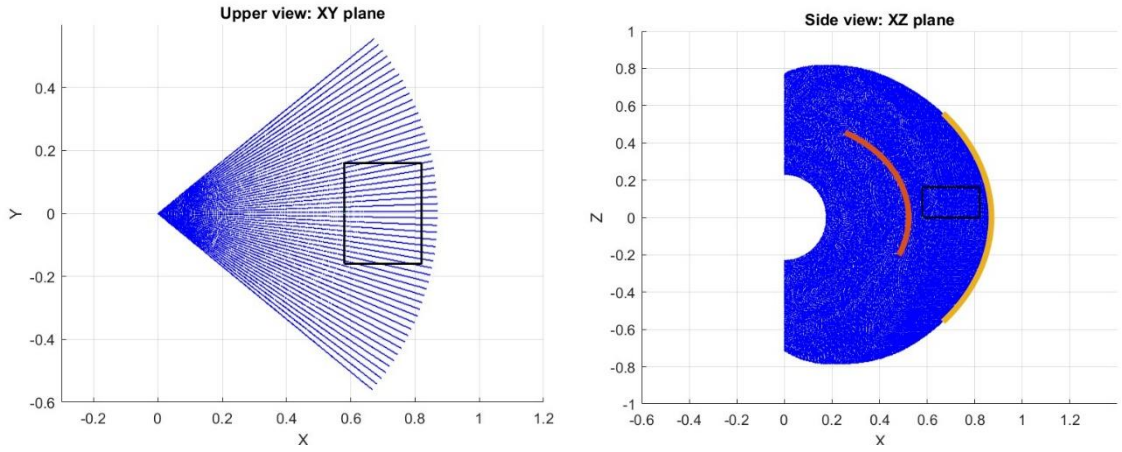


Figure 5 Montecarlo workspace

2 Motion Planning

First, we must define the maximum angular velocities and the maximum angular accelerations/decelerations. We choose these values arbitrarily.
These limits directly determine the minimum achievable task execution time.

	Max ang. vel. $[rad/s]$	Max ang. acc./dec. $[\mathrm{rad/s^2}]$
Joint 1	100	4
Joint 2	100	3
Joint 3	100	4
Joint 3	200	5

Now, for each consecutive pair of trajectory points we compute the minimum actuation time required by each actuator.
The time to move between these two points is set to the maximum of the four rise times, which guarantees synchronous arrival at the target configuration.
Joints that would arrive earlier are slowed down.

2.1. Trajectory n.1: Lines and Parabolas method

The Lines and Parabolas method defines the joint trajectory in segments composed of straight lines and parabolic arcs. A critical issue of this approach is that the resulting accelerations appear as abrupt steps rather than smooth curves. This piecewise definition prevents capturing realistic transients, which is undesirable since motors experience gradual transitions between on and off states. Consequently, the method may lead to unrealistic dynamic behaviour during motion.

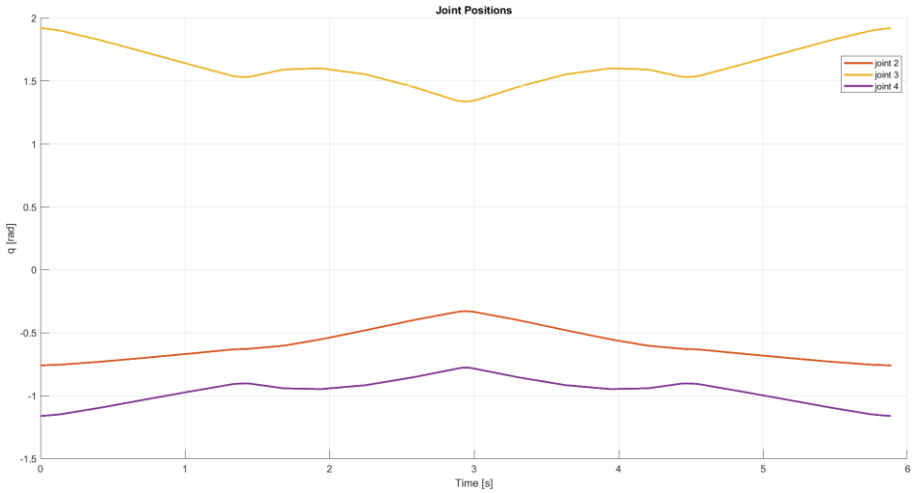


Figure 6 Joint Rotation

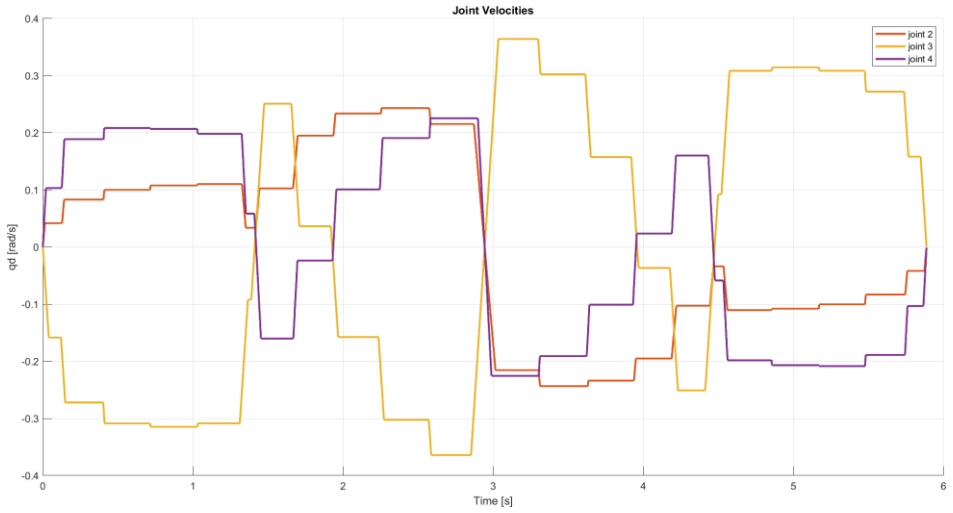


Figure 7 Joints Velocities

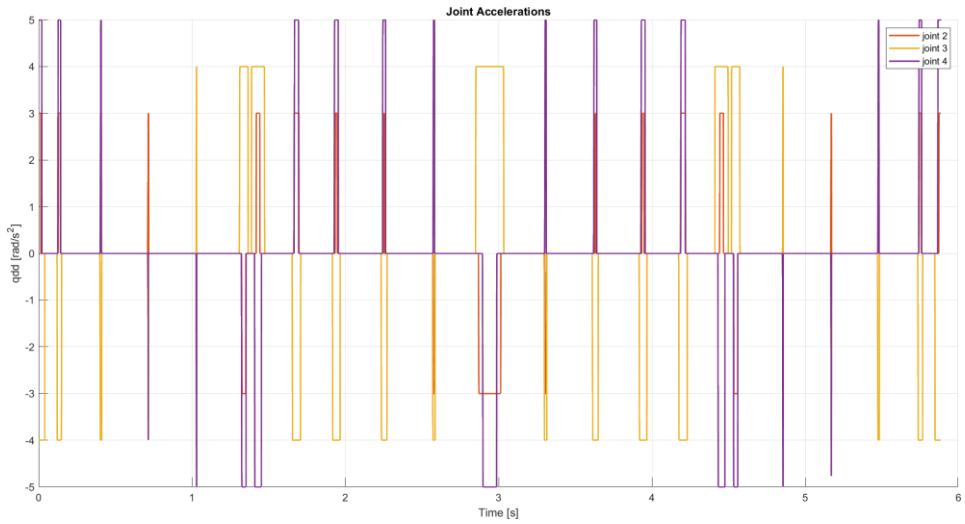


Figure 8 Joints Accelerations

2.2. Trajectory n. 2: Cubic Spline Method

The Cubic Spline method divides the trajectory into multiple intervals. In the first and last intervals, a 5th-order spline is used to ensure that both acceleration and velocity at the start and end points are zero. This approach produces piecewise continuous accelerations rather than abrupt steps, resulting in a motion profile that is more realistic and easier for motors to follow. Additionally, the spline method allows smooth transitions between segments and providing better control over trajectory continuity. Overall, it represents a more practical and physically feasible solution compared to the "Lines and Parabolas" method.

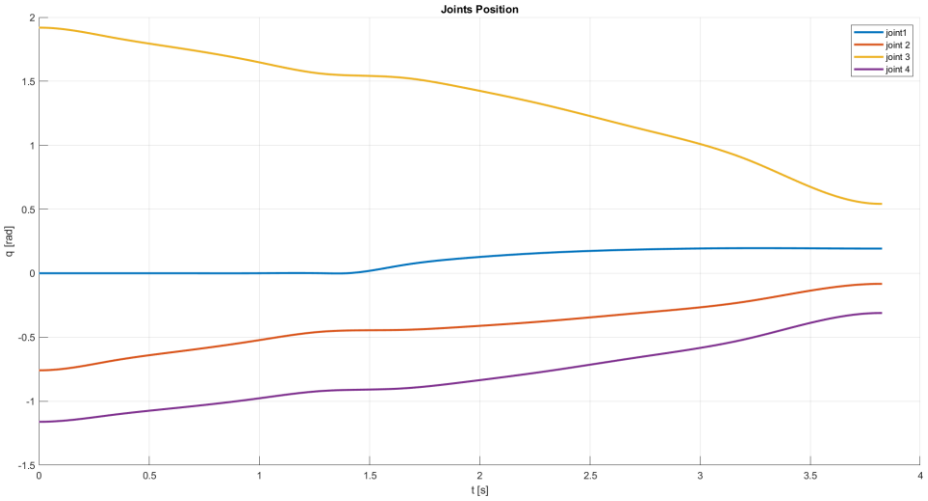


Figure 9 Joints Rotation

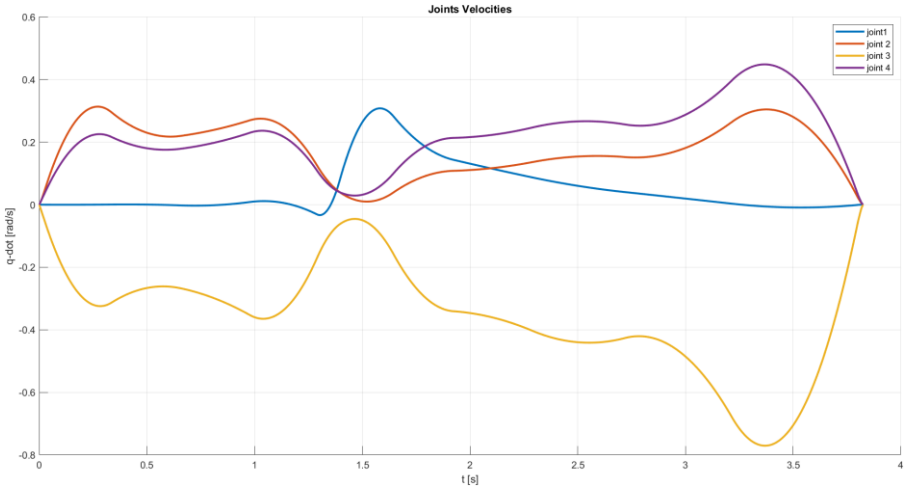


Figure 10 Joints Velocities

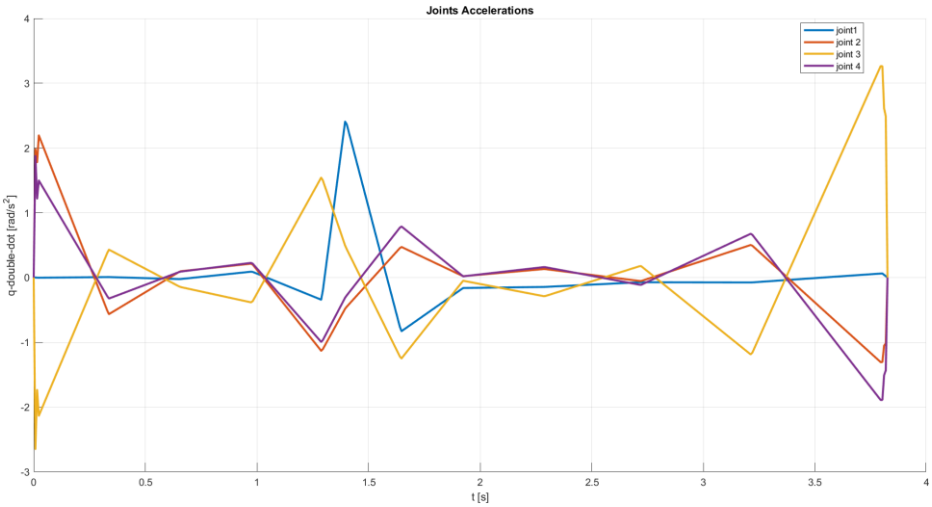


Figure 11 Joints Accelerations

Using 5th-order splines at the trajectory boundaries can introduce unwanted oscillations. To minimize this effect, the first and last intervals were chosen to be very short, with points located very close together. This limits the oscillations while still ensuring zero velocity and acceleration at the start and end.

2.3. Convalidation through Direct Kinematic

We use the direct kinematics to verify if the trajectory in output from the motion planning approximate in a good way the real trajectory.

We obtain this trajectory:

- Trajectory n. 1

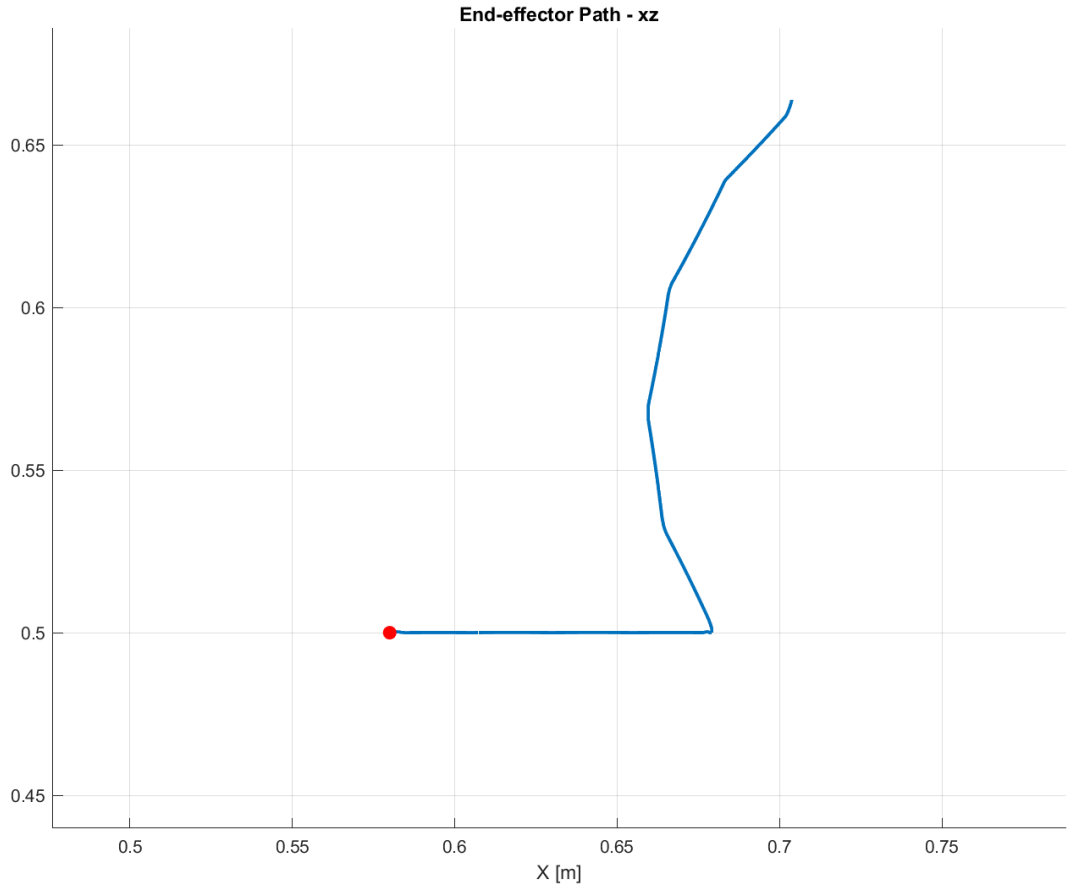


Figure 12 Trajectory 1 - Lines and Parabolas - with direct kinematics

- Trajectory n. 2

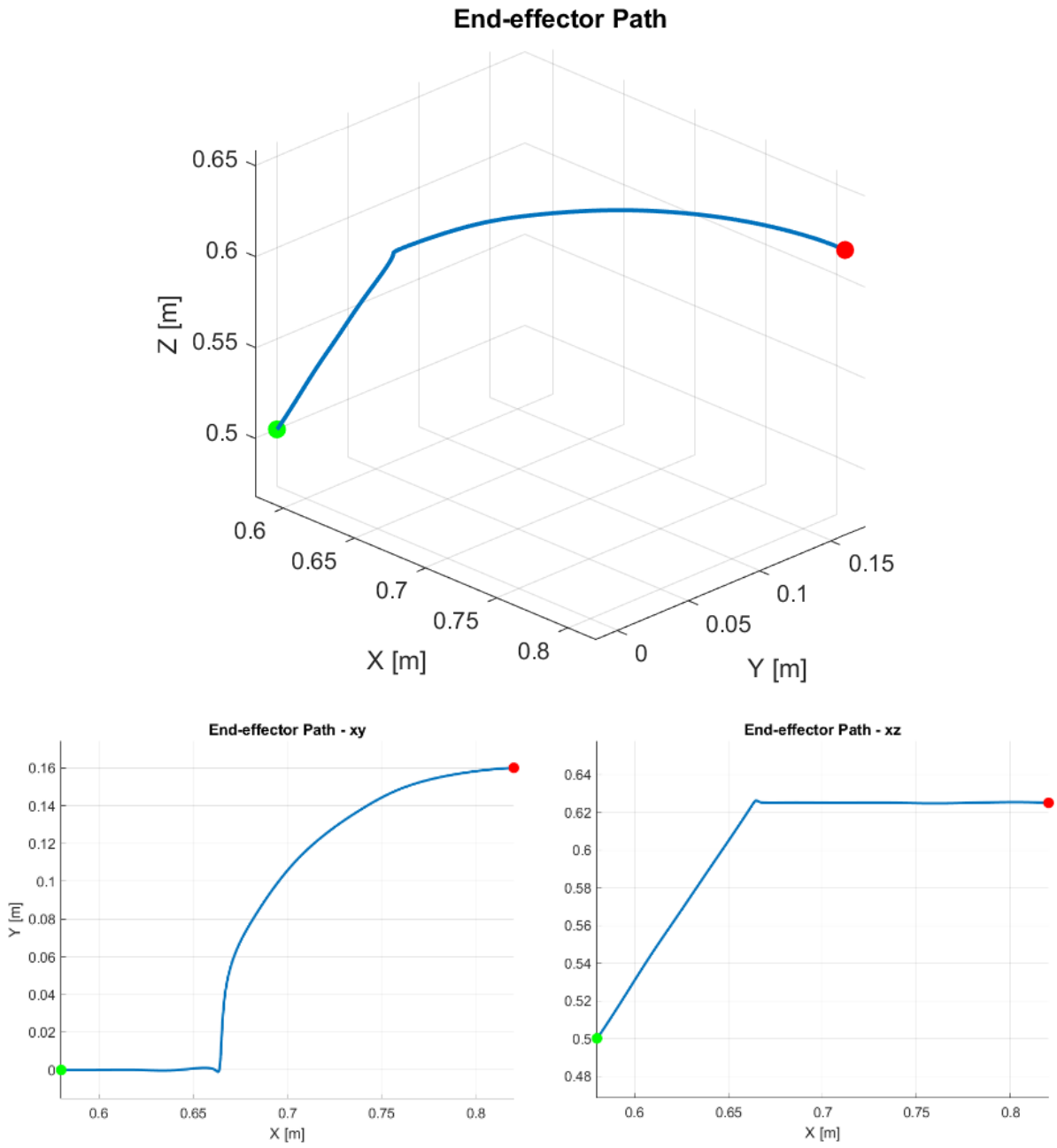


Figure 13: Trajectory 2 - Cubic Splines - with direct kinematics

As we can see both the trajectory are well approximated with the motion planning strategies.

3 Dynamics analysis

The evaluation of the 4DOF robot dynamics lies in the calculation of the forces, torques and inertias, that are acting internally or externally on the robot. The evaluation of such forces is important for the design and sizing of the mechanical parts and motors. To do so, an extended kinematic problem must be solved, extending the functions and the matrices defined earlier

3.1. Torques with MATLAB

First, we import for each trajectory the curves of rotation, velocity and accelerations for each joint. Then we define the masses, the inertias, the lengths and the centres of mass (COM) of the arms (provided by the CAD model).

The vectors links length and centres of mass (COM):

$$\mathbf{L} \in \mathbb{R}^{4\times1}, \qquad \mathbf{G} \in \mathbb{R}^{4\times1}$$

The system's mass–inertia matrix is defined as a diagonal matrix:

$$\mathbf{M} = \mathrm{diag}(m, m, m, m_4, m_4, m_4, J_4, m_3, m_3, m_3, J_3, m_2, m_2, m_2, J_2, J_1).$$

The external force vector:

$$\mathbf{F}_{\mathrm{ext}}^{\mathsf{T}} = [0,\ 0,\ -mg,\ 0,\ 0,\ -m_4 g,\ 0,\ 0,\ 0,\ -m_3 g,\ 0,\ 0,\ 0,\ -m_2 g,\ 0,\ 0]$$

We can now start to evaluate torque.

The Cartesian acceleration of all system points is:

$$\ddot{x} = \dot{J}(q, \dot{q})\,\dot{q} + J(q)\,\ddot{q}$$

The inertial forces in Cartesian space are computed:

$$F_{\mathrm{si}} = -M\ddot{x}$$

representing Newton's second law applied to all 16 coordinates of the multibody system.

The total force acting on the system is:

$$F_s = F_{\mathrm{ext}} + F_{\mathrm{si}}$$

The joint torques are obtained by projecting Cartesian forces to joint space through the **transpose Jacobian**:

$$\tau = -J(q)^{\top} F_s$$

3.1.1. Jacobian matrices J - numerical

The Jacobian is computed **numerically** using **central finite differences**.

We prefer to use the numerical one because it's faster than the symbolical one, we do the same for the derivative one.

For each joint j at each instant i :

1. Perturb the joint angle forward (e_j *is the unitary vector*):

$$Q^{+} = Q + \delta\, e_j$$

2. Perturb backward:

$$Q^{-} = Q - \delta\, e_j$$

3. Compute extended forward kinematics with both perturbed values ($S_e(Q^{+})$ *and* $S_e(Q^{-})$).
4. Approximate each Jacobian column as:

$$J_{:,j} = \frac{S_e(Q^{+}) - S_e(Q^{-})}{2\delta}$$

$$J \in \mathbb{R}^{16\times 4\times i}$$

At the end we have to assemble all the Jacobian and the results is:

$J \in \mathbb{R}^{16\times 4\times n}$, where n is the number of instants

3.1.2. Time derivative of the Jacobian matrices $\dot{J}$ - numerical

The time derivative of the Jacobian is computed numerically using a **directional difference along the joint-velocity direction**.

Since:

$$\dot{J} = \frac{d}{dt} J(Q(t)) = \frac{\partial J}{\partial Q}\dot{Q}$$

Approximating it by evaluating the Jacobian at two configurations displaced along $\dot{Q}$:

1. Forward step:

$$Q^+ = Q + h\dot{Q}$$

2. Backward step:

$$Q^- = Q - h\dot{Q}$$

3. Compute both Jacobians $J^+$and J^-

$$J^+ = J(Q^+), \qquad J^- = J(Q^-)$$

4. Approximate:

$$\dot{J} \approx \frac{J^+ - J^-}{2h}$$

$$\dot{J} \in \mathbb{R}^{16\times4\times i}$$

At the end we have to assemble all the derivative of the Jacobian and the results is:

$\dot{J} \in \mathbb{R}^{16\times4\times n}$, where n is the number of instants

This avoids second-order derivatives and keeps the computation stable.

Both h and δ are chosen equal to 1e-4.

3.1.3. Results

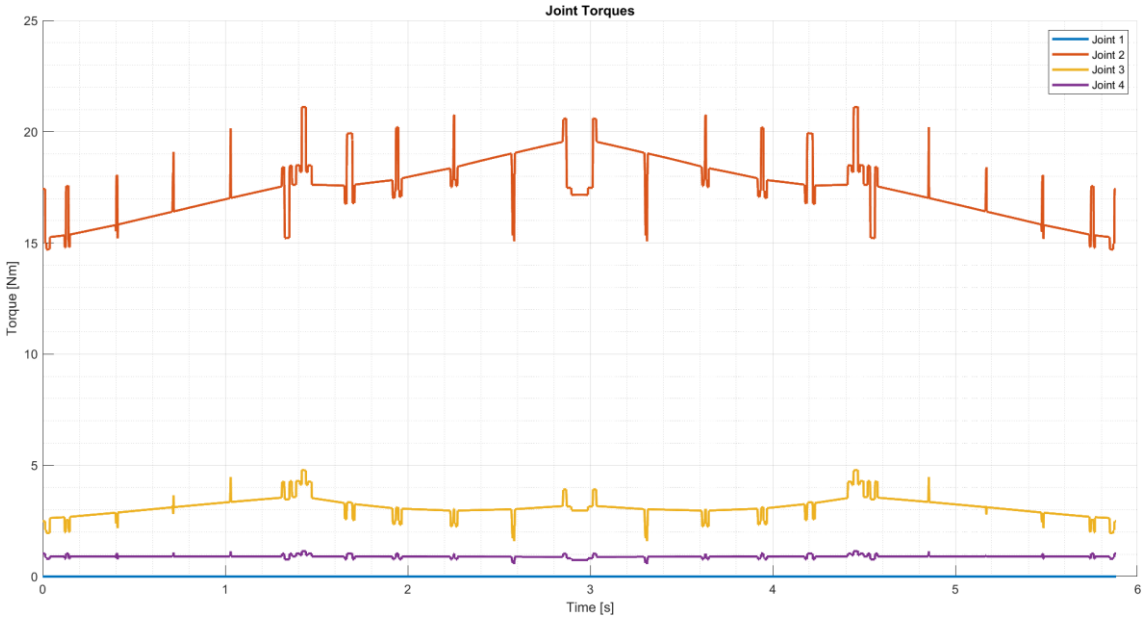


Figure 14: Torques - Trajectory 1

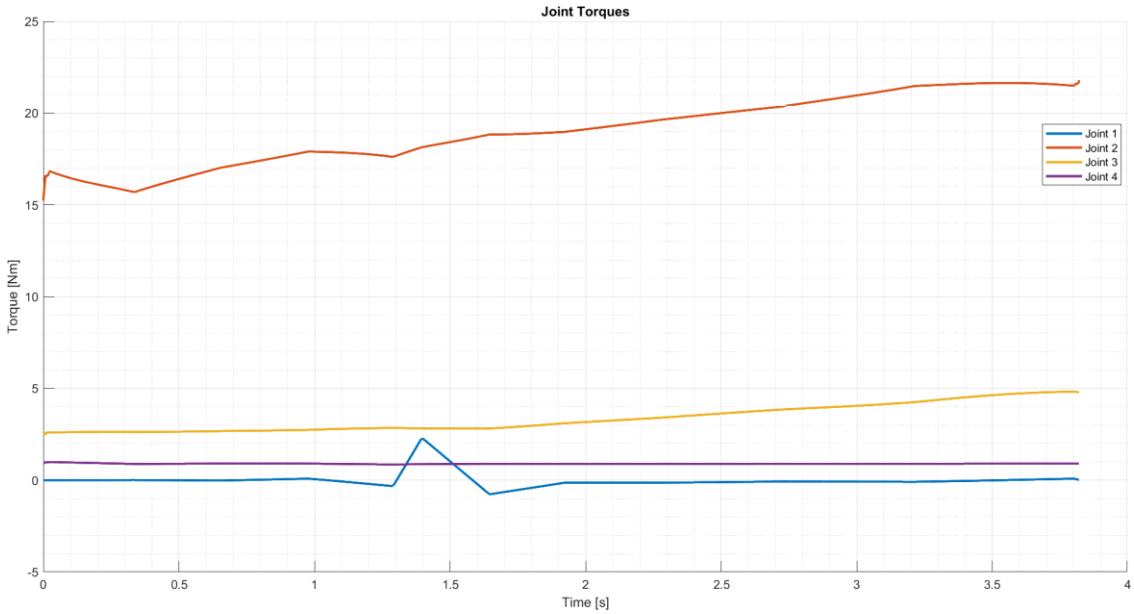


Figure 15: Torques - Trajectory 2

In the first trajectory torque profiles, clear peaks appear at the transitions between linear segments and parabolic blends.
This behaviour is expected, because in a **lines-and-parabolas** trajectory the acceleration is zero along the straight segments and takes a **constant non-zero value** only inside the parabolic intervals.
These sudden changes in acceleration generate corresponding jumps in the required torques, producing the observed peaks exactly where the parabolic phases occur.

3.2. SIMSCAPE Model

For the SIMSCAPE model we export the CAD file and import it in MATLAB. We add a mass of 0.5 Kg at the end of the end effector. For each joint we provide in input the relative curves (rotation, velocity and acceleration) obtained with the motion planning and the software compute the torques.

This are the torques obtained compared with the ones computed with numerical method:

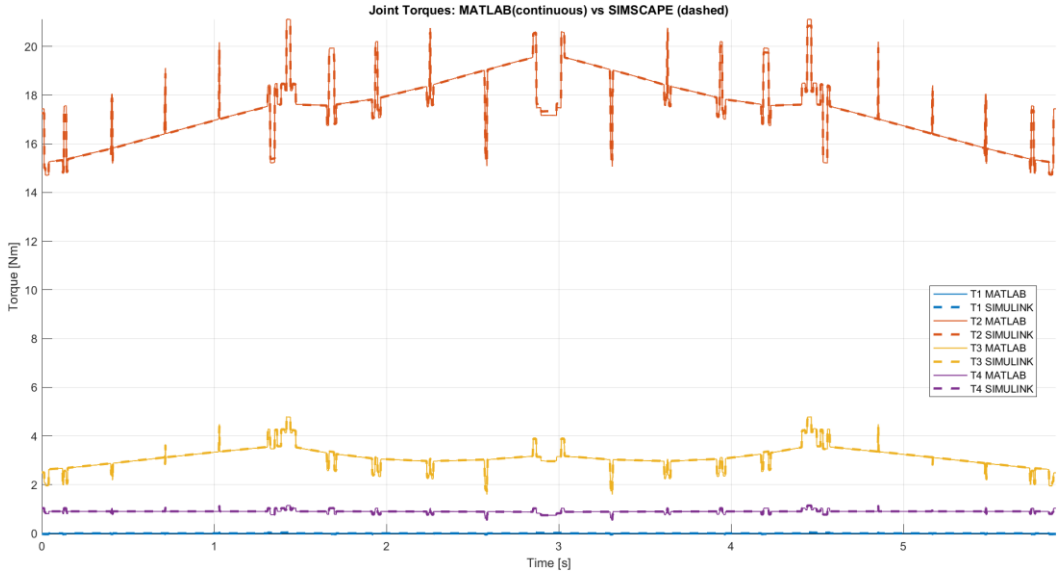


Figure 16 : Torques - Trajectory 1 - MATLAB vs SIMSCAPE

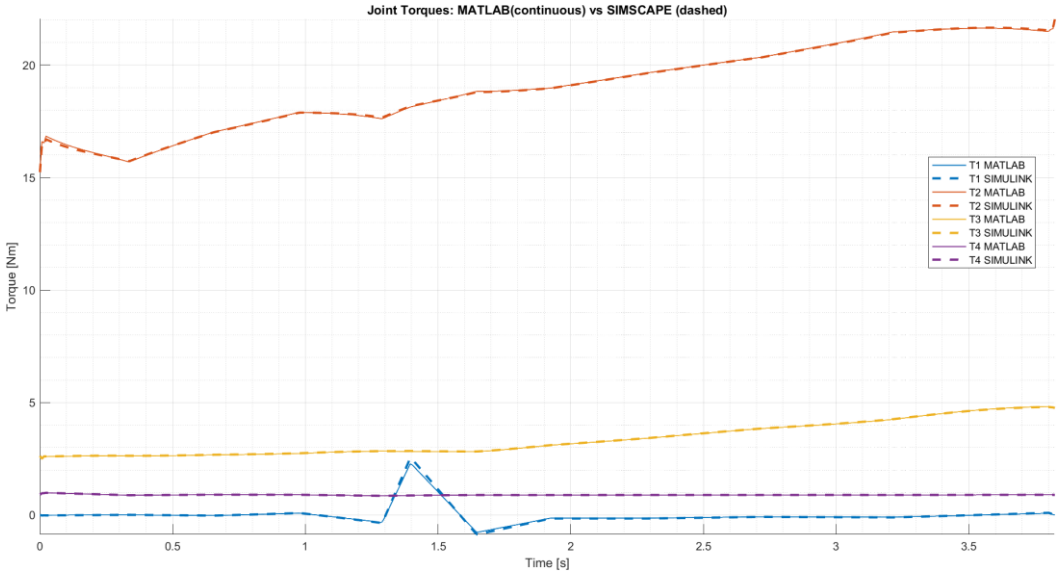


Figure 17 : Torques - Trajectory 2 - MATLAB vs SIMSCAPE

With this plot we can appreciate that the Simscape and MATLAB torques are basically the same.

3.3. Different Payload

We repeat what we did previously but changing the hanged mass. We use these values [0, 1] Kg. What we obtain are this torques.

For the first trajectory:

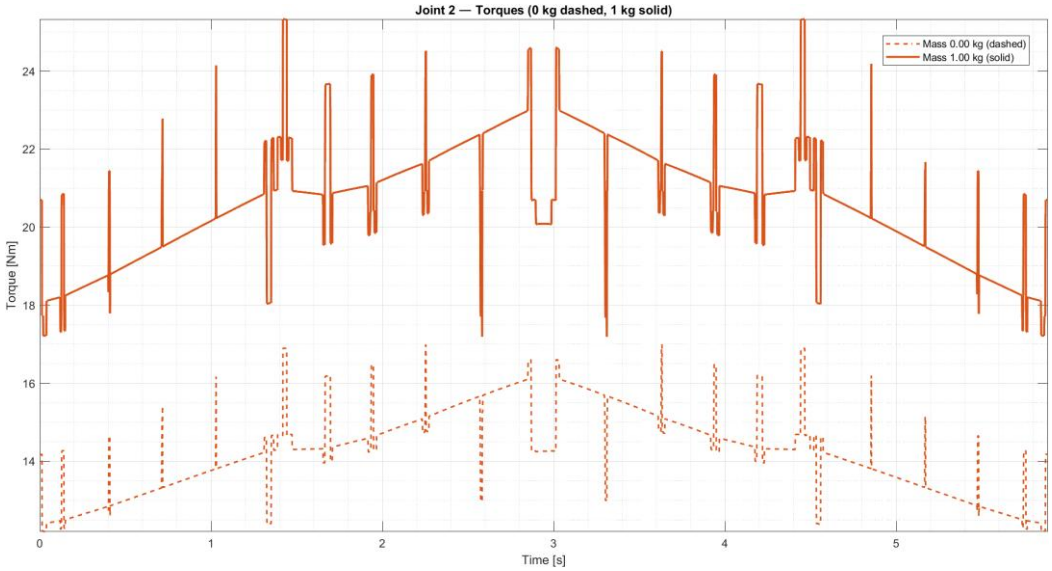


Figure 18: Joint 2 Torque - Trajectory 1 - 0 Kg vs 1 Kg

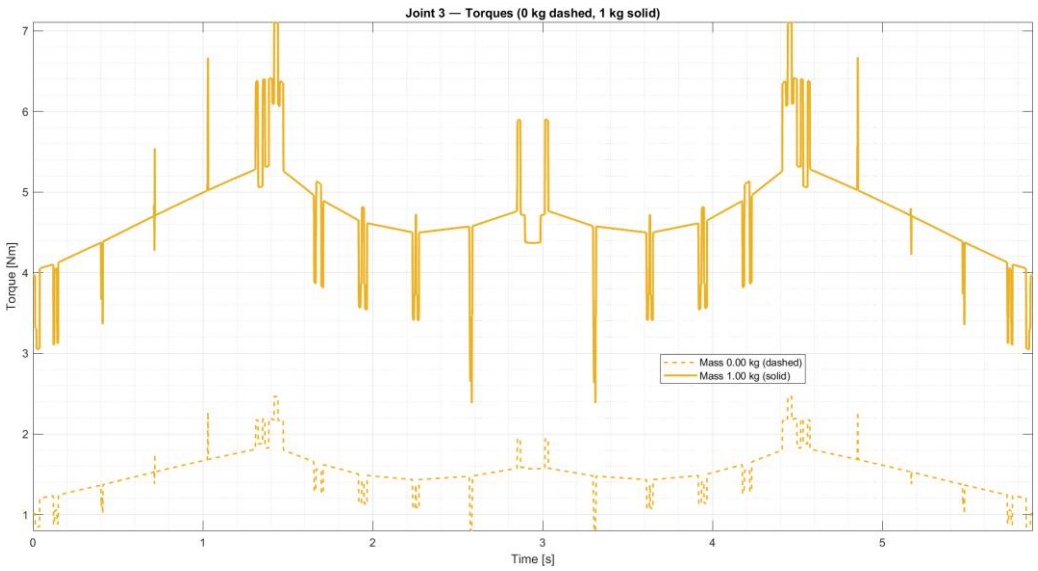


Figure 19: Joint 3 Torque - Trajectory 1 - 0 Kg vs 1 Kg

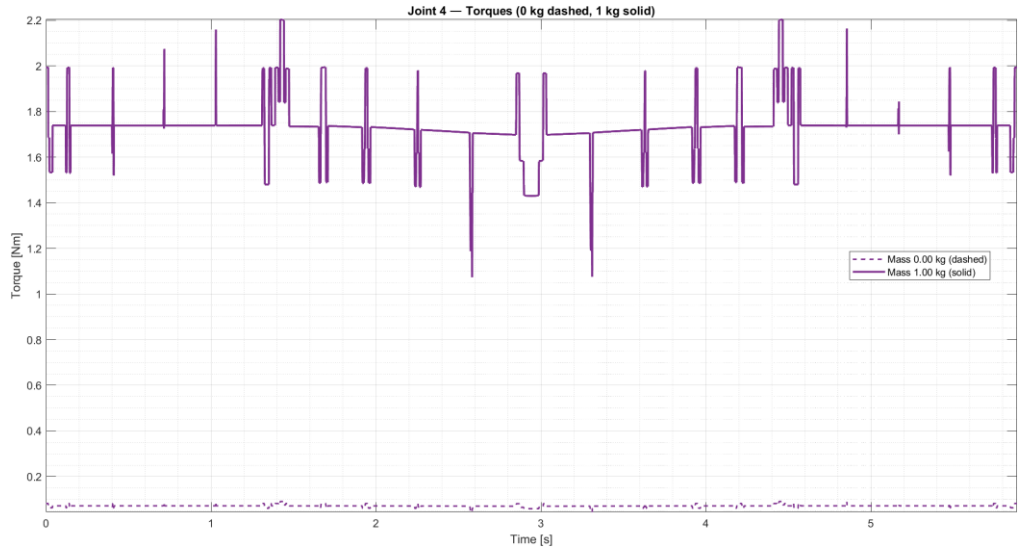


Figure 20: Joint 4 Torque - Trajectory 1 - 0 Kg vs 1 Kg

We avoid the plot of the first joint because is always zero.

For the second trajectory:

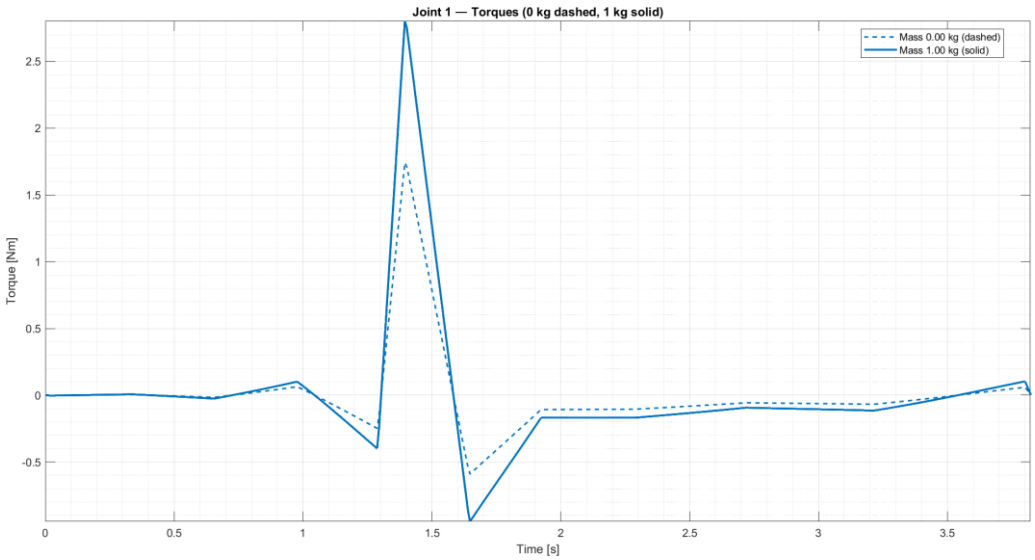


Figure 21: Joint 1 Torque - Trajectory 2 - 0 Kg vs 1 Kg

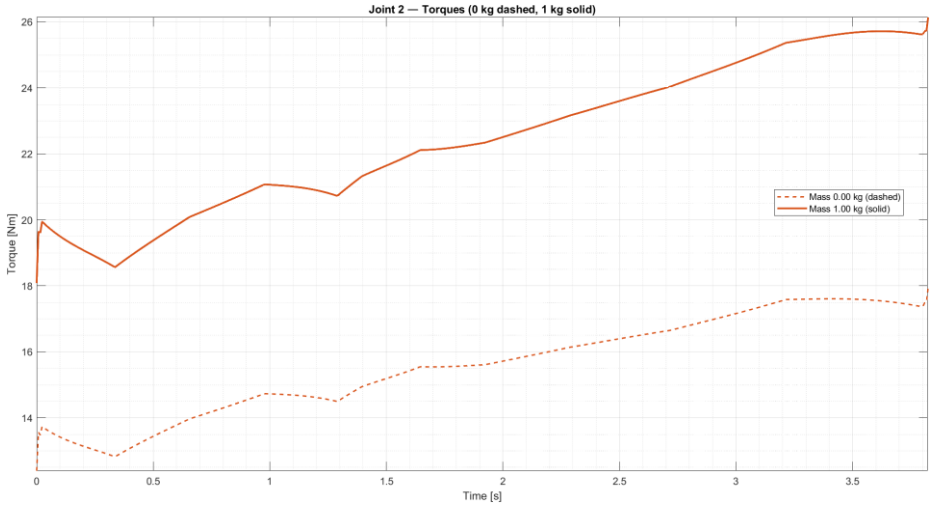


Figure 22: Joint 2 Torque - Trajectory 2 - 0 Kg vs 1 Kg

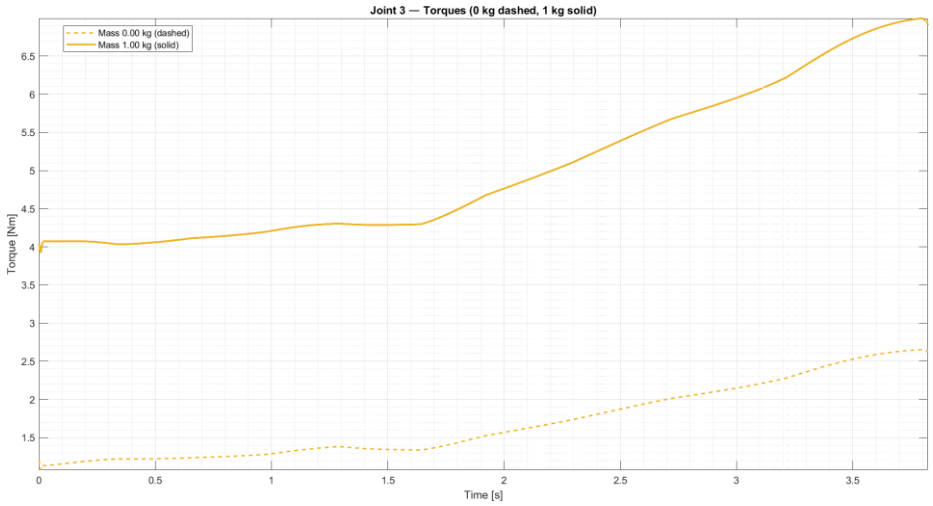


Figure 23: Joint 3 Torque - Trajectory 2 - 0 Kg vs 1 Kg

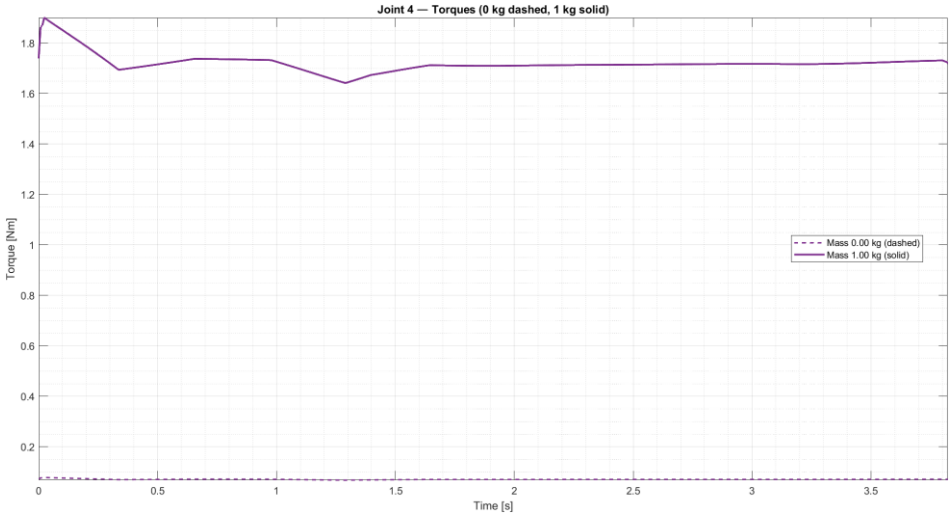


Figure 24: Joint 3 Torque - Trajectory 2 - 0 Kg vs 1 Kg

4 Motor and transmission sizing

Let's see the process for motors and transmissions sizing.

First for each joint and trajectory we must define:

- the **root-mean-square (RMS)** values of the required load torques C_r and joints acceleration $\ddot{q}$ are computed
- the **optimal transmission ratio** (τ_{opt}):

$$\tau_{\text{opt}} = \sqrt{\frac{J_m \, \ddot{q}_{\text{rms}}}{C_{\text{r,rms}}}}$$

- the **parameters** α and β:

$$\alpha = \frac{C_n^2}{J_m}, \quad \beta = 4 \, J_r \, \ddot{q}_{\text{rms}}^2$$

- the **maximum and minimum feasible transmission ratio** (τ_{max} and τ_{min})

$$\tau_{\text{max}} = \frac{\sqrt{J_m}(\sqrt{\alpha} + \sqrt{\alpha - \beta})}{2 J_r \ddot{q}_{\text{rms}}}, \qquad \tau_{\text{min}} = \frac{\sqrt{J_m}(\sqrt{\alpha} - \sqrt{\alpha - \beta})}{2 J_r \ddot{q}_{\text{rms}}}$$

Then we choose a transmission ratio for each joint.
At the end we must evaluate for each joint if the chosen transmission ratio (τ) and motor satisfy this constrains:

- **Nominal torque condition**

$$C_{m,n} > \tau \, C_{\text{r,rms,max}}$$

- **Maximum torque condition**

$$C_{m,\text{max}} > \tau \, C_{\text{max}}$$

- **Transmission ratio limits**

$$\tau_{\text{max}} > \tau > \tau_{\text{vel}} > \tau_{\text{min}}$$

$$\tau_{\text{vel}} = \frac{\dot{q}_{\text{max}}}{\dot{q}_{p,m,\text{max}}}$$

These are the values of α and β for each joint for both the trajectories

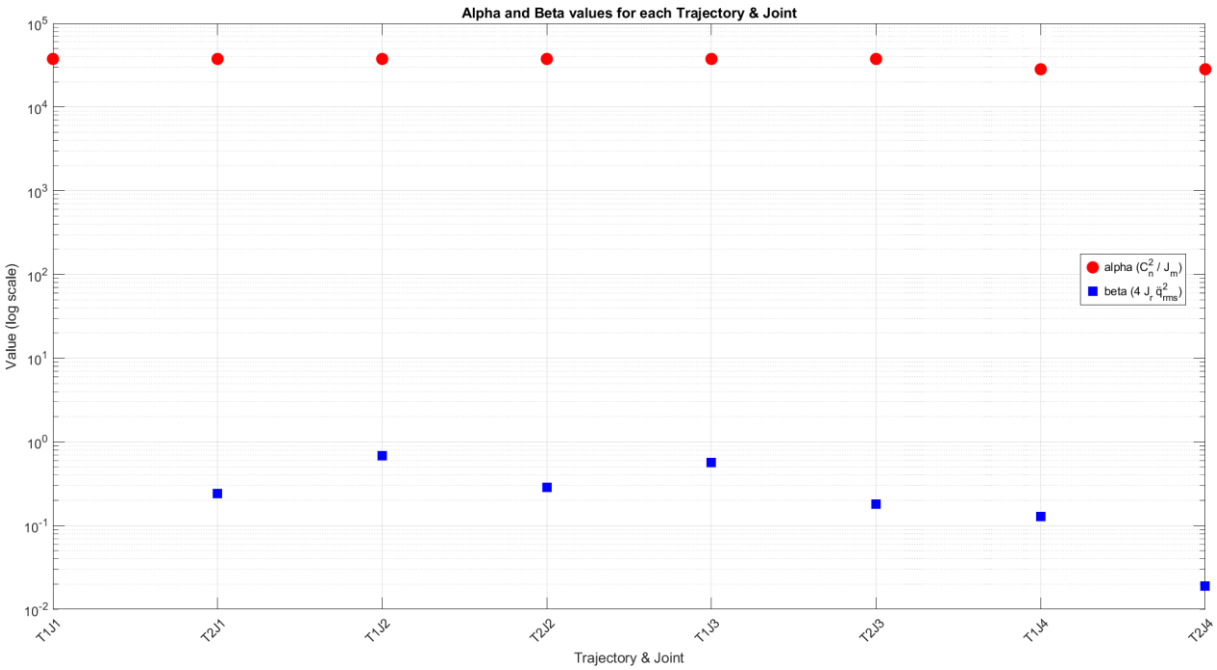


Figure 25 α and β values - J for joint and T for trajectory

We can appreciate that $\alpha >> \beta$ and $J_m \approx 0$, so:

$$\left(\sqrt{\alpha} - \sqrt{\alpha - \beta}\right) \approx 0 \quad => \quad \tau_{\min} \approx 0$$

$$\left(\sqrt{\alpha} + \sqrt{\alpha - \beta}\right) \approx 2\sqrt{\alpha} \quad => \quad \tau_{\max} \approx \frac{Cn}{J_r \ddot{q}_{\text{rms}}}$$

This means that motor and transmission sizing can be carried out under static conditions; however, to achieve a more accurate evaluation, we also perform a dynamic analysis.

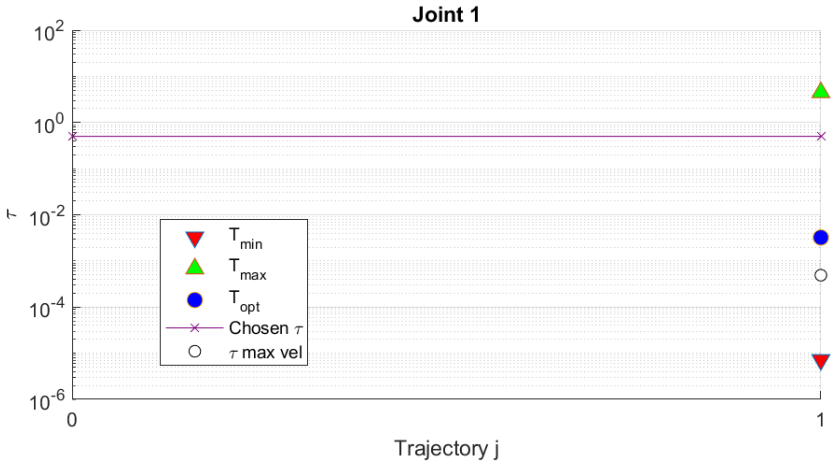
4.1. Motor sizing for standard payload

In this section we analyse joint by joint if the chosen motors and transmissions satisfy the 3 conditions.

For each joint we also must satisfy the requirements for both the trajectory. In the plots "trajectory 1" is associated at 0 and "trajectory 2" is associated to 1.
For each joint $C_{\text{r, rms, max}}$ is the maximum between $C_{\text{r, rms, trajectory1}}$ and $C_{\text{r, rms, trajectory2}}$, same thing for C_{max}.

Let's analyse each joint.

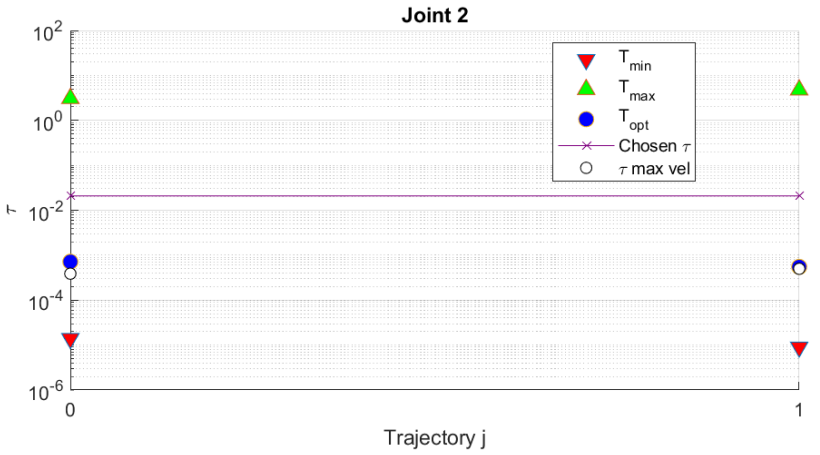
- **Joint 1**
  Motor: HG-MR23B, 200W
  Transmission: $\tau_1 = 0.5$



Condition 1: $C_{m,n} = 0.64 > \tau\, C_{\text{r, rms, max}} = 0.2208$ → Passed

Condition 2: $C_{m,max} = 1.9 > \tau\, C_{\text{max}} = 1.2628$ → Passed

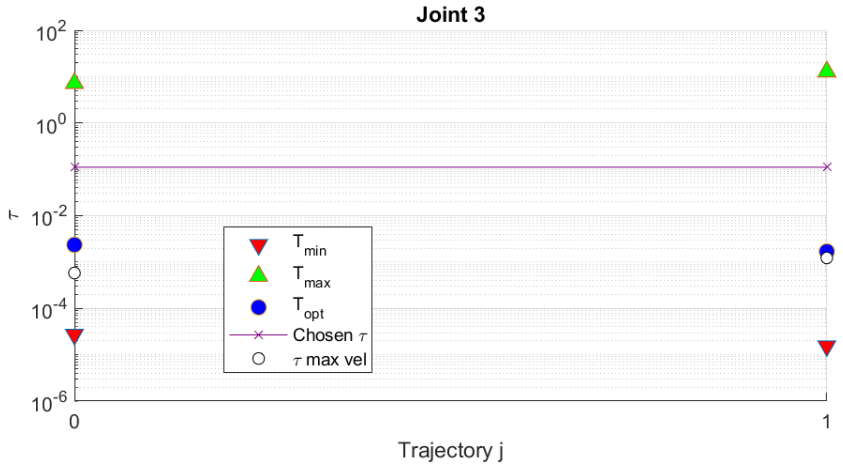
- **Joint 2**
  Motor: HG-MR23B, 200W
  Transmission: $\tau_2 = 0.02$



Condition 1: $C_{m,n} = 0.64 > \tau\, C_{\text{r, rms, max}} = 0.3983$ → Passed

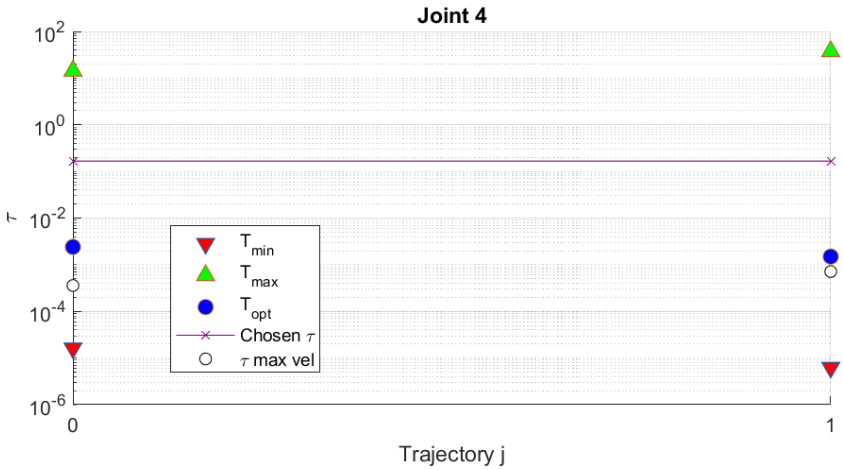
Condition 2: $C_{m,max} = 1.9 > \tau\, C_{\text{max}} = 0.4602$ → Passed

- **Joint 3**
  Motor: HG-MR23B, 200W
  Transmission: $\tau_3 = 0.11$



Condition 1: $C_{m,n} = 0.64 > \tau\, C_{\text{r, rms, max}} = 0.3824$ → Passed
Condition 2: $C_{m,max} = 1.9 > \tau\, C_{\text{max}} = 0.5337$ → Passed

- **Joint 4**
  Motor: HG-MR13B, 100W
  Transmission: $\tau_4 = 0.33$



Condition 1: $C_{m,n} = 0.32 > \tau\, C_{\text{r, rms, max}} = 0.1503$ -> Passed
Condition 2: $C_{m,max} = 0.95 > \tau\, C_{\text{max}} = 0.1910$ -> Passed

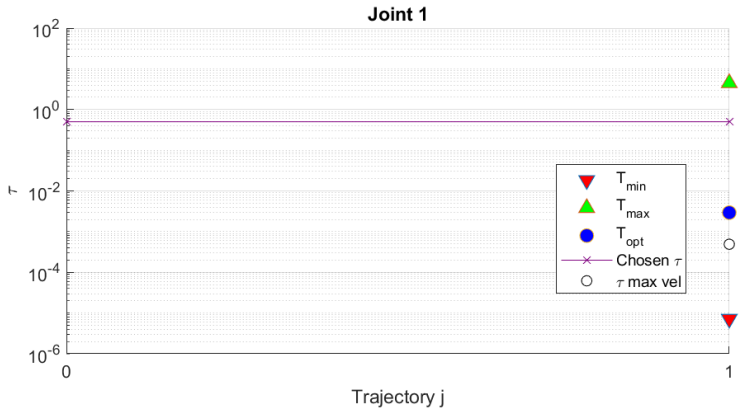
So, the transmission ratio and the motor chosen are feasible with the dynamics of the system with a payload of 0.5 Kg.

4.2. Sizing with different payload

We just evaluate the worst case when the payload is equal to 1 Kg, for sure for payload with lower mass we will have better results.

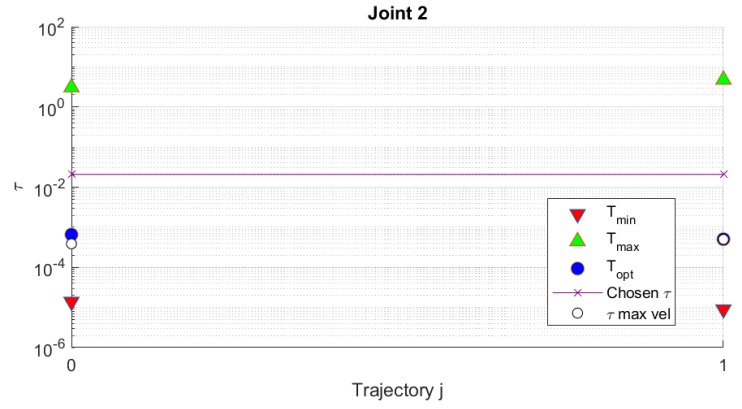
These are the results with a mass of 1 Kg using the same transmission ratio and motor:

- **Joint 1**



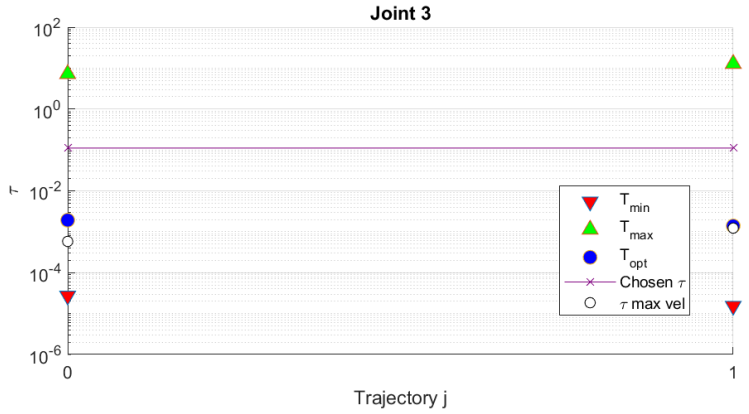
Condition 1: $C_{m,n} = 0.64 > \tau\, C_{\text{r, rms, max}} = 0.2671$ → Passed
Condition 2: $C_{m,max} = 1.9 > \tau\, C_{\text{max}} = 1.5283$ → Passed

- **Joint 2**



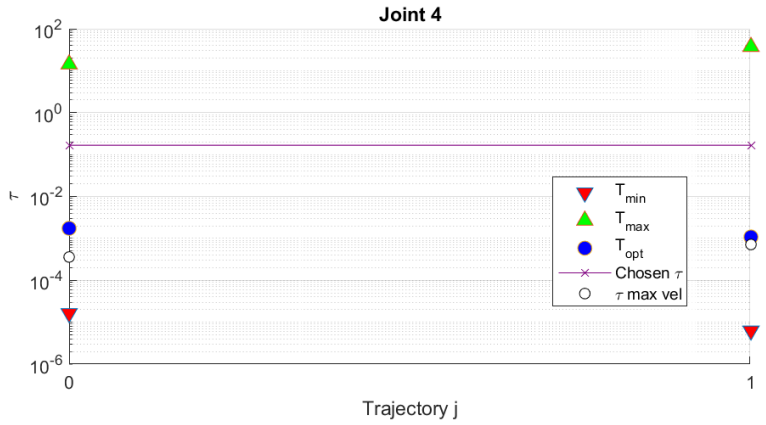
Condition 1: $C_{m,n} = 0.64 > \tau\, C_{\text{r, rms, max}} = 0.4703$ → Passed
Condition 2: $C_{m,max} = 1.9 > \tau\, C_{\text{max}} = 0.5455$ → Passed

- **Joint 3**



Condition 1: $C_{m,n} = 0.64 > \tau\, C_{\text{r, rms, max}} = 0.5694$ → Passed
Condition 2: $C_{m,max} = 1.9 > \tau\, C_{\text{max}} = 0.7815$ → Passed

- **Joint 4**



Condition 1: $C_{m,n} = 0.64 > \tau\, C_{\text{r, rms, max}} = 0.2887$ → Passed
Condition 2: $C_{m,max} = 1.9 > \tau\, C_{\text{max}} = 0.3669$ → Passed

5 Possible Improvement

Our idea is placing a torsional spring on the second joint to compensate the high masses of the arms. Using a torsional spring we can modify the value of the $C_{\mathrm{r,\,rms}}$ and our goal is to decrease it.

$$\tau_{\mathrm{opt}} = \sqrt{\frac{J_m\,\ddot{q}_{\mathrm{rms}}}{C_{\mathrm{r,rms}}}}$$

If the value of $C_{\mathrm{r,\,rms}}$ decrease, τ_{opt} will increase.

We choose a spring of 0.6 N*m/deg.

The torques on the second joint with the torsional spring are:

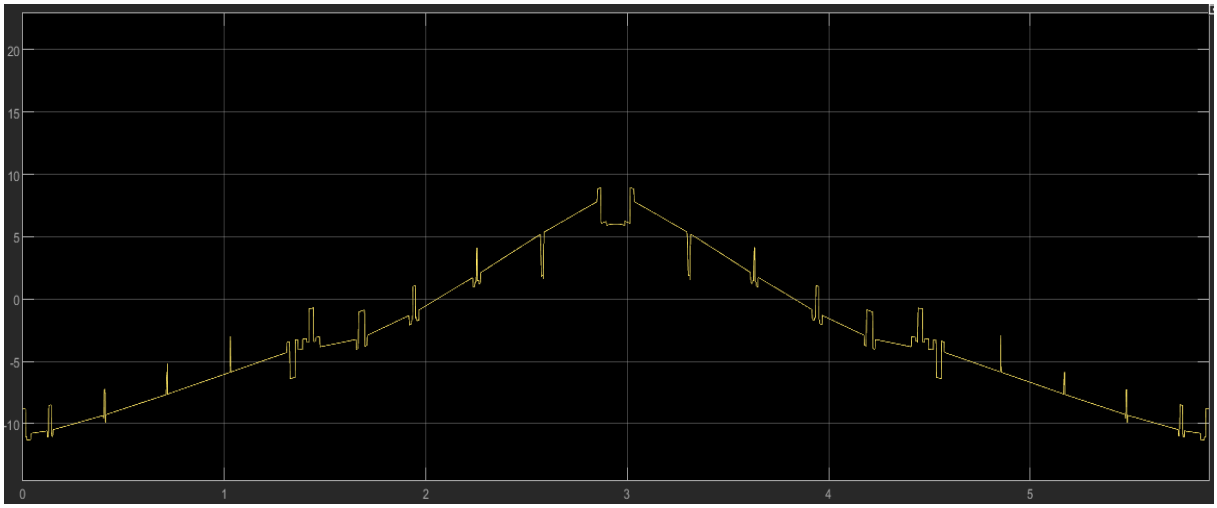


Figure 26 Joint 2 Torque with spring - Trajectory 1

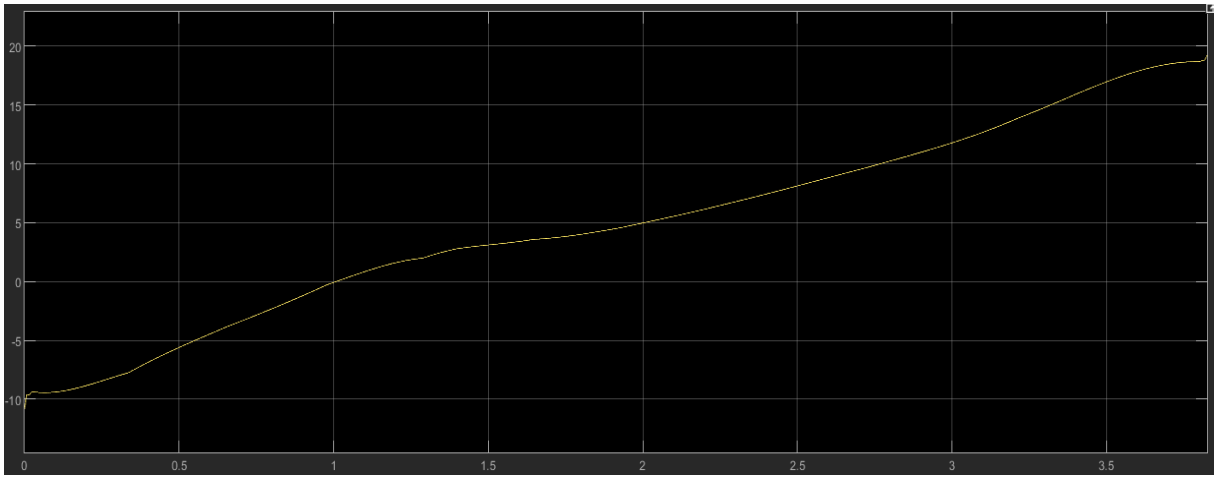


Figure 27 Joint 2 Torque with spring - Trajectory 2

Let's compare the second joint characteristics with and without torsional spring:

	With spring		Without Spring	
	Trajectory 1	Trajectory 2	Trajectory 1	Trajectory 2
$C_{\text{r, rms}}$ [Nm]	6.17	9.3	17.48	19.12
τ_{opt}	0.00120	0.00079	0.00071	0.00055

As we can see the values of τ_{opt} increase and now are nearest the value of $\tau_2 = 0.02$. For sure the two conditions on nominal and maximal torques are satisfied because both the $C_{\text{r, rms}}$ values and the C_{max} values for both the trajectories decrease.

Notice that the other parameters $\alpha, \beta, \tau_{\text{max}}, \tau_{\text{min}}$ and τ_{vel} don't change.

We can also dimension springs for the other joints; we just focus on the second joints because we have very high torque for both the trajectories.

6 Quality of the robot

To describe the quality of the robot under analysis there are different possible approaches. The one developed in this report is the one related to the manipulability ellipses of force and velocity and the corresponding conditioning index.

In order to find these ellipses, different ingredients are needed:

- Kinetostatic duality:

$$\begin{cases} F_q = -J^T F_s \\ \dot{S} = J\dot{Q} \end{cases}$$

- Set of joint velocities and forces of constant norm:

$$\begin{cases} \dot{Q}^T \dot{Q} = 1 \\ F_q^T F_q = 1 \end{cases}$$

Combining those two sets of relations we can obtain the following system:

$$\begin{cases} \dot{S}^T (JJ^T)^{-1} \dot{S} = 1 \\ F_S^T (JJ^T) F_s = 1 \end{cases}$$

These relations permit to express the ellipses associated to the so called Manipulability matrix:

$$M = JJ^T$$

A first characteristic that can be noticed is that the manipulability ellipses in force and velocity are orthogonal, in fact the eigenvectors of M and M^{-1} are coincident but the corresponding eigenvalues are inverse.

As said, the ellipses, for this 4DOF robot in particular are ellipsoid (3D), can be used to describe the quality of a mechanism. The closer the ellipses are to a circle (sphere in the space) the higher is the manipulability and so the "quality" of the robot in a specific point of its trajectory. If the ellipse tends to a circle the point in which the system is working is called "isotropic point" and in this configuration a homogeneity of behaviour in all the directions is reached. Instead if the ellipse tends to degenerate in a segment means that the robot is close to a singularity and so its behaviour will be asymmetric.

To build the ellipsoids and so define their volume it's necessary to evaluate the length of the semi-axis, that mathematically correspond to the singular values of the Jacobian matrix or the square root of the eigenvalues of the matrix M (remember that the eigenvalues associated to the velocity ellipses are the inverse of the ones of the force ellipses described below).

$$\sigma_i = \lambda(J) \;\; or \;\; \sigma_i = \sqrt{\lambda(JJ^T)}$$

However, the only analysis of the ellipsoids and their volume, that depends on the Jacobian matrix, does not represent an absolute comparable value, especially close to the singularity, because this depends on the dimensions of the system.

To overcome this problem, the conditioning number is defined as the ratio between the maximum and the minimum singular values of the Jacobian matrix. This number is unitary when an isotropic point is reached and tends to infinity near a singularity. That is why usually (in order to work in a range between 0 and 1) the inverse of the conditioning number is used. Values close to 0 means singularity, instead close to 1 means isotropy. Usually, a value above 0.2 represents acceptable working conditions.

After this brief theoretical recap about the manipulability ellipses and why they were chosen to describe the quality of a robot, now the results about the analysed mechanism are reported.

6.1. Ellipsoids of force

Below are some frames about the trend of the ellipsoids along the two trajectories:

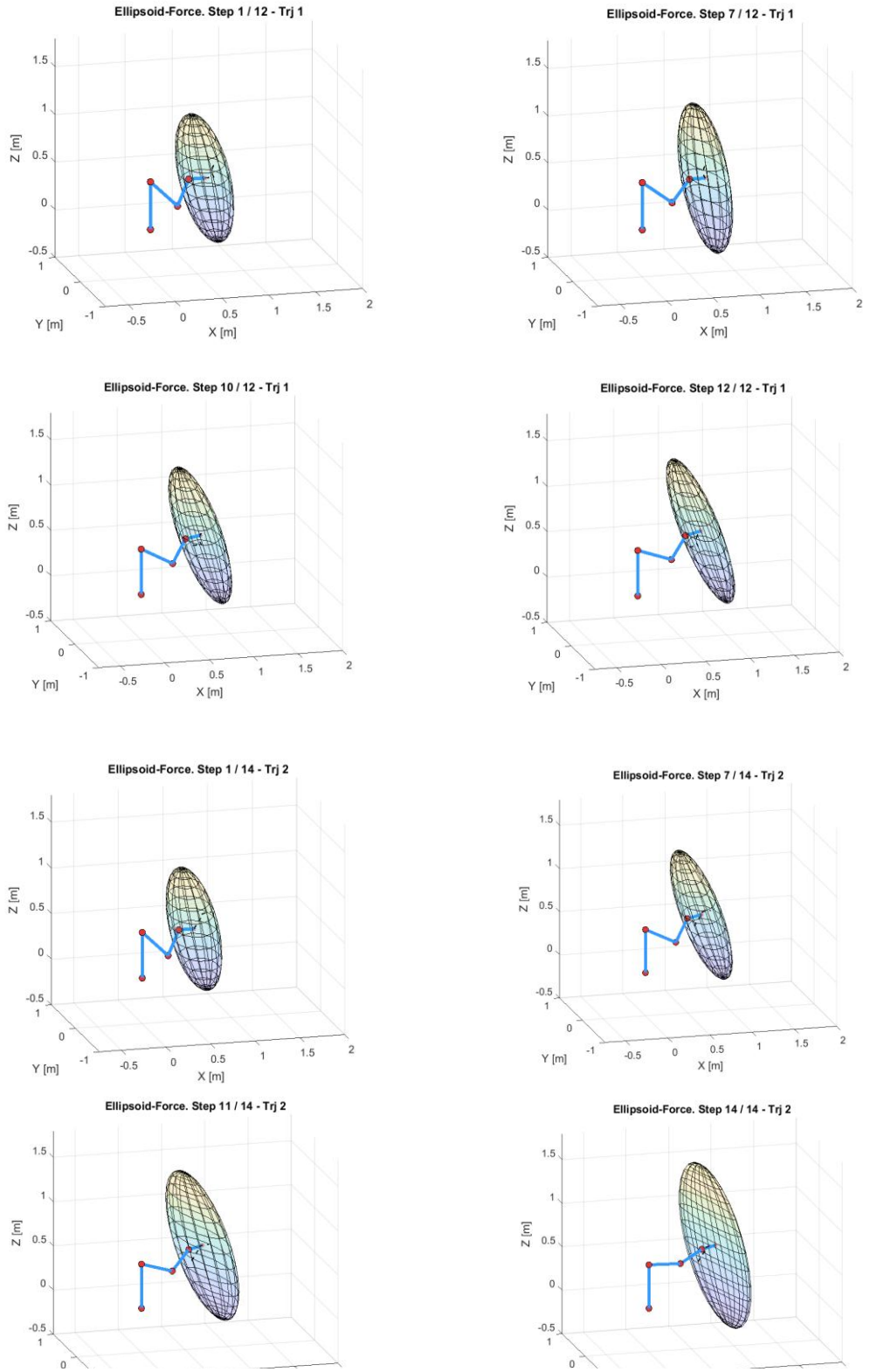


Figure 28 Force Ellipsoids

From those frames it can be seen the evolution of the ellipsoids and how, as the robot elongates in the space in both the trajectory, they tend to assume a very compressed shape that means the we are reaching points close to a singularity configuration.
This behaviour is confirmed also by the evaluation of the inverse conditioning number:

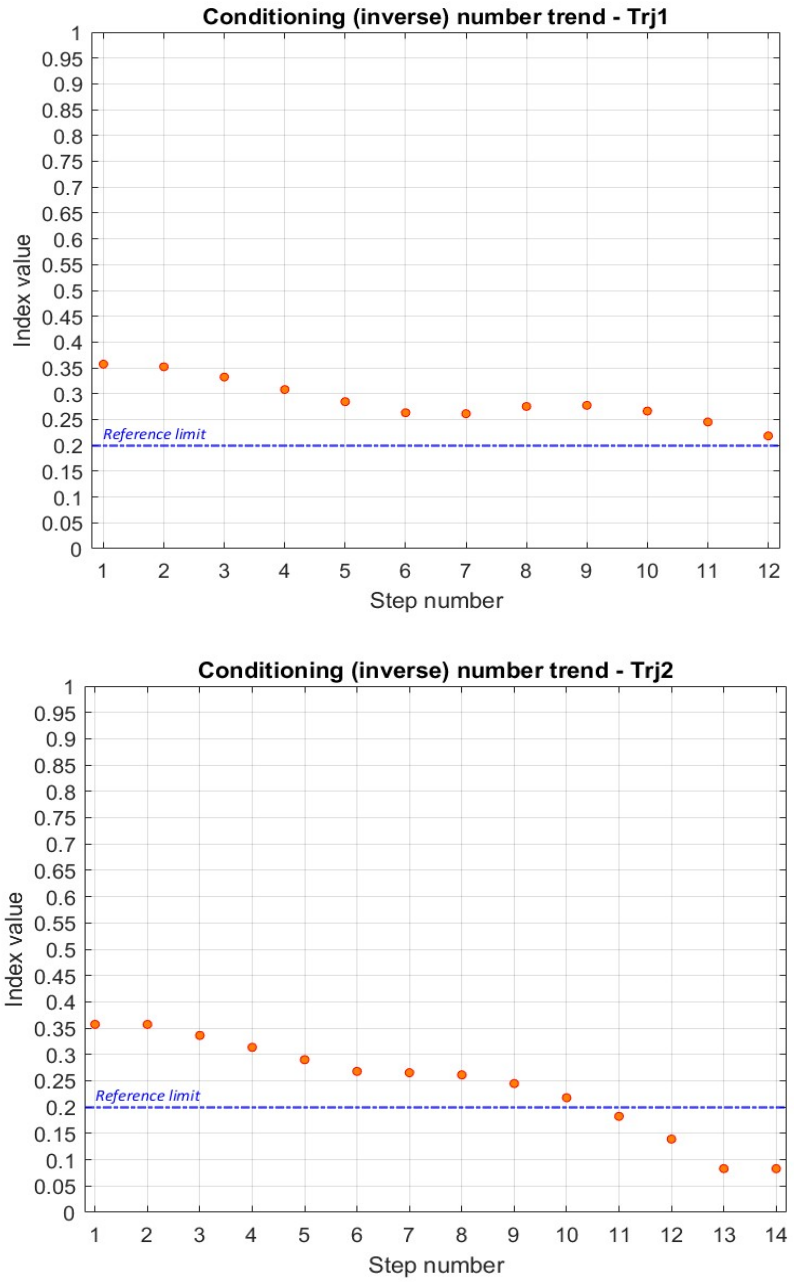


Figure 29 Condition Number

From those diagrams it's visible that, along the trajectory 1 the robot is working always over the value 0.2, so on acceptable conditions. Instead along the trajectory 2, on the lasts points, the conditioning number it's under the reference 0.2, so it means that we are reaching a singularity and so the accuracy of the robot could be compromised.

6.2. Ellipsoids of velocity

Below are reported some frames showing the evolution of the ellipsoids of velocity along the two trajectories, noticing that as expected they are orthogonal to the ones of force previously discussed:

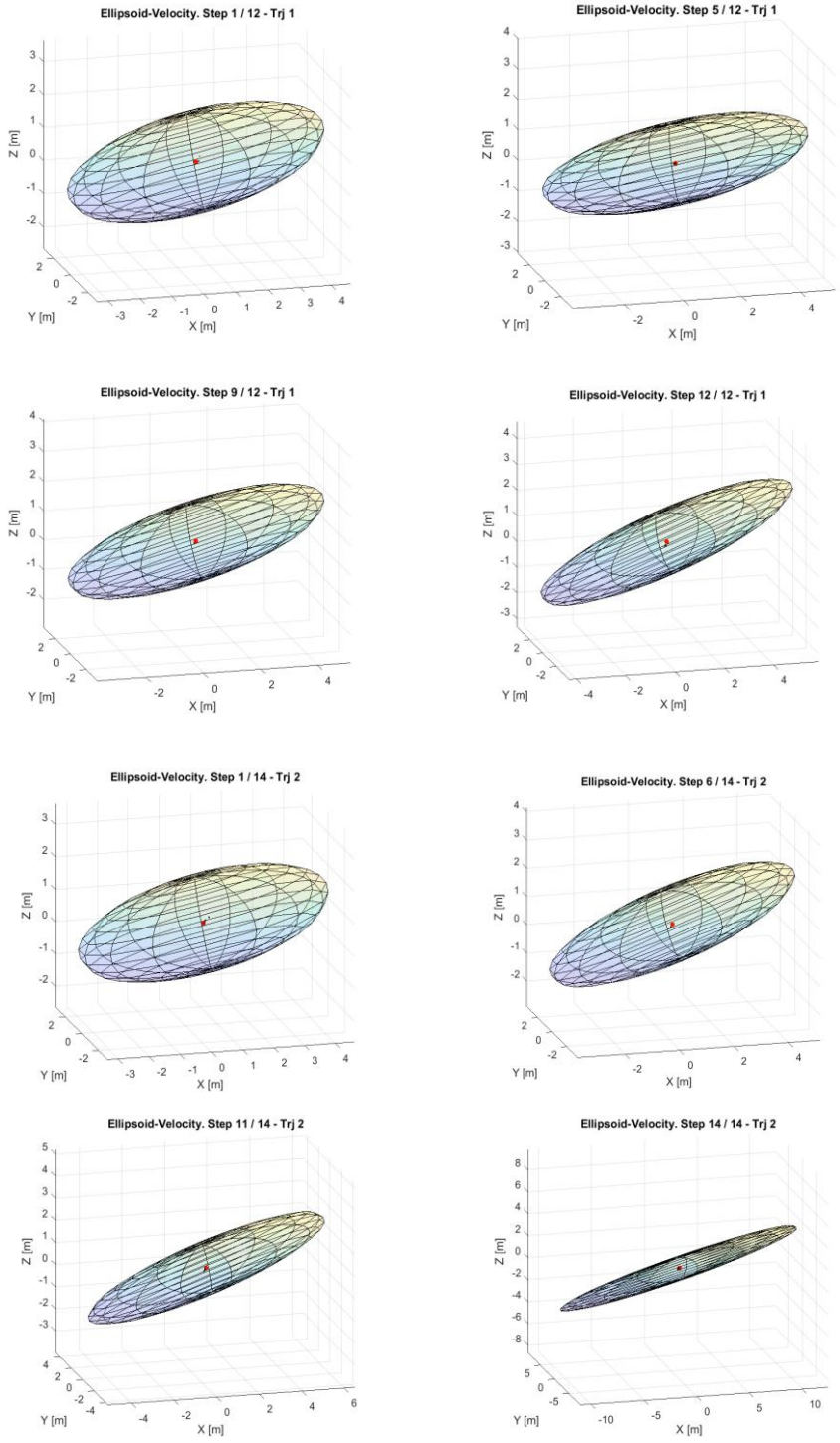


Figure 30 Velocity Ellipsoids

Also, in this case it can be seen how the ellipsoids evolve in terms of orientation and shape and how, more accentuated along the second trajectory, there is a tendency to reach a singularity.

As before, also in this case are reported the diagrams about the conditioning number to confirm the results observable from the ellipsoids behaviour:

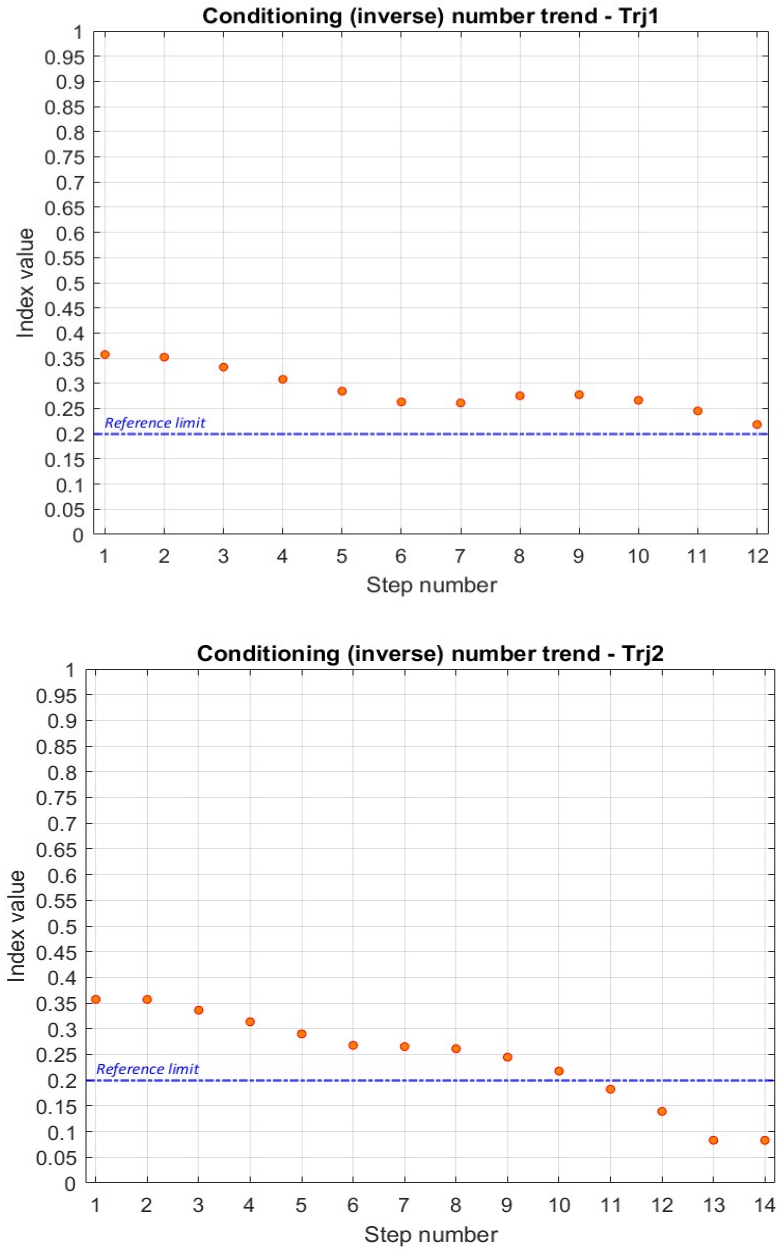
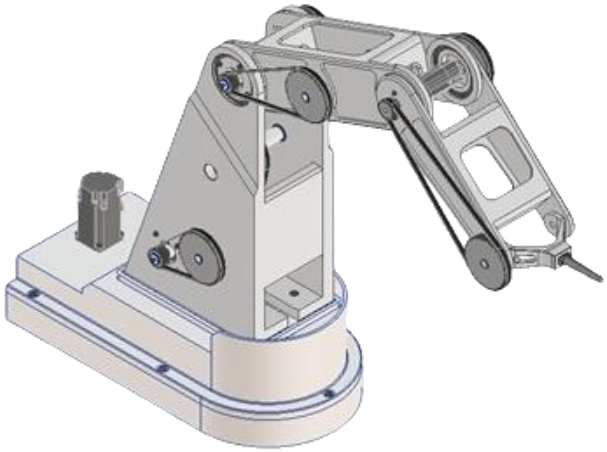


Figure 31 Conditioning Number

These diagrams simply confirm, what we expect. The inverse conditioning number is the same independently from which type of ellipsoid we consider. If the quality of the robot is not good in terms of velocity in a certain point, the correspondent ellipsoid of force will be only the orthogonal one, but it will also show in any case that we are near to a singularity.

7 Robot model – CAD

The arms of the robot are in ABS; the base and the shafts are in aluminium.



7.1. Assembly

First, we assemble the second arm. Notice that the third motor is fixed at the second arm.

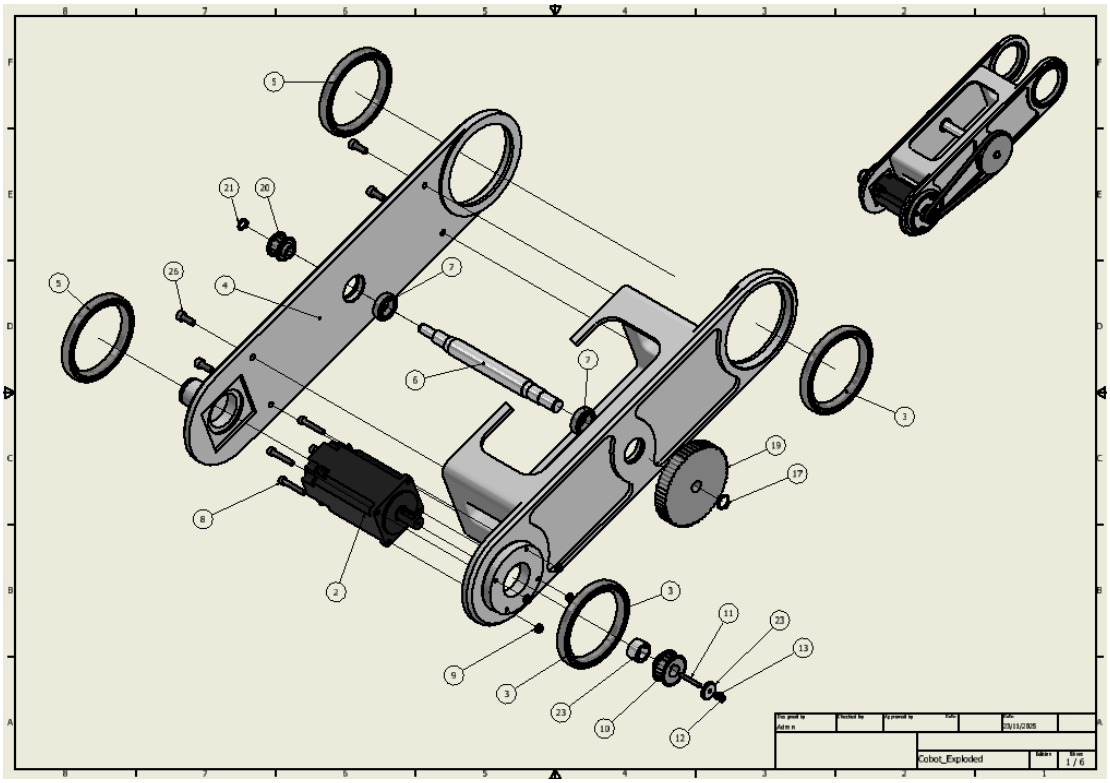


Figure 32 Assembly second arm

Then, we assemble the third and the end effector

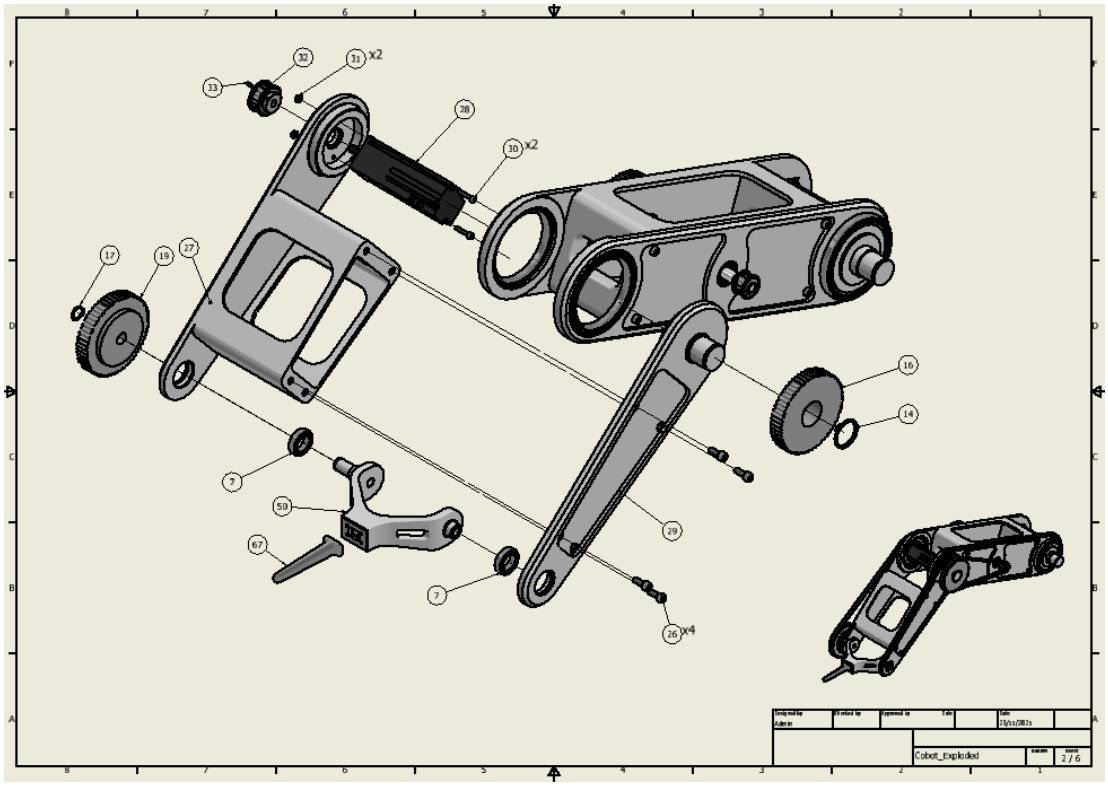


Figure 33 Assembly second arm and end effector(EE)

This is the section view of the two arm and end-effector:

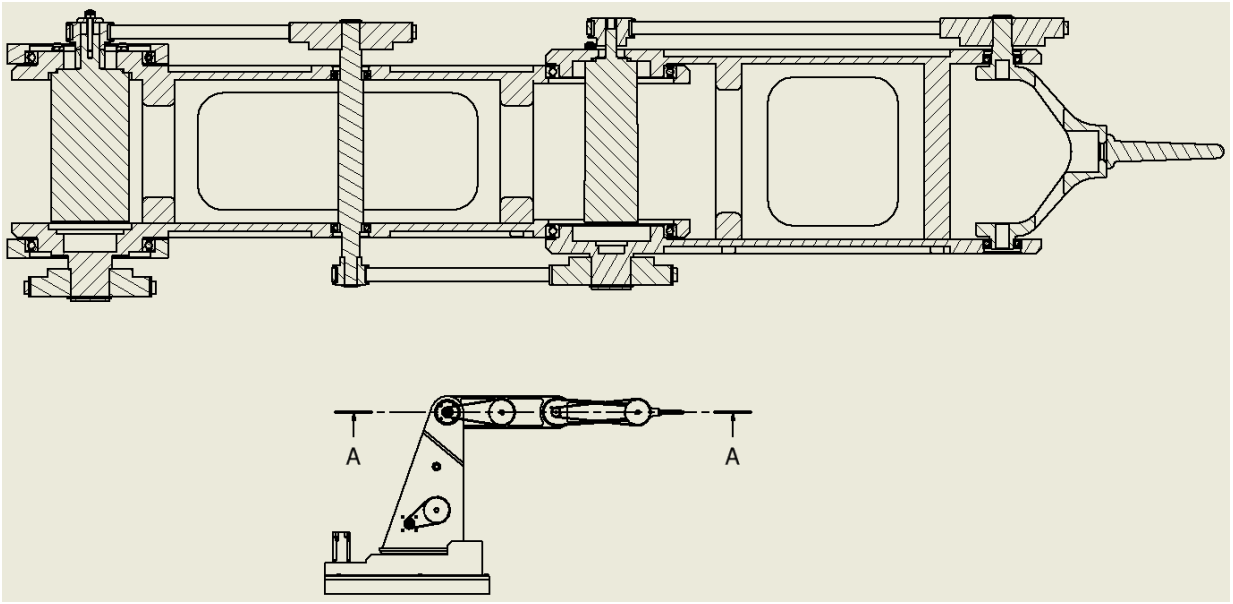


Figure 34 Top section view of the second and third arms and EE

Now we can assemble the first arm:

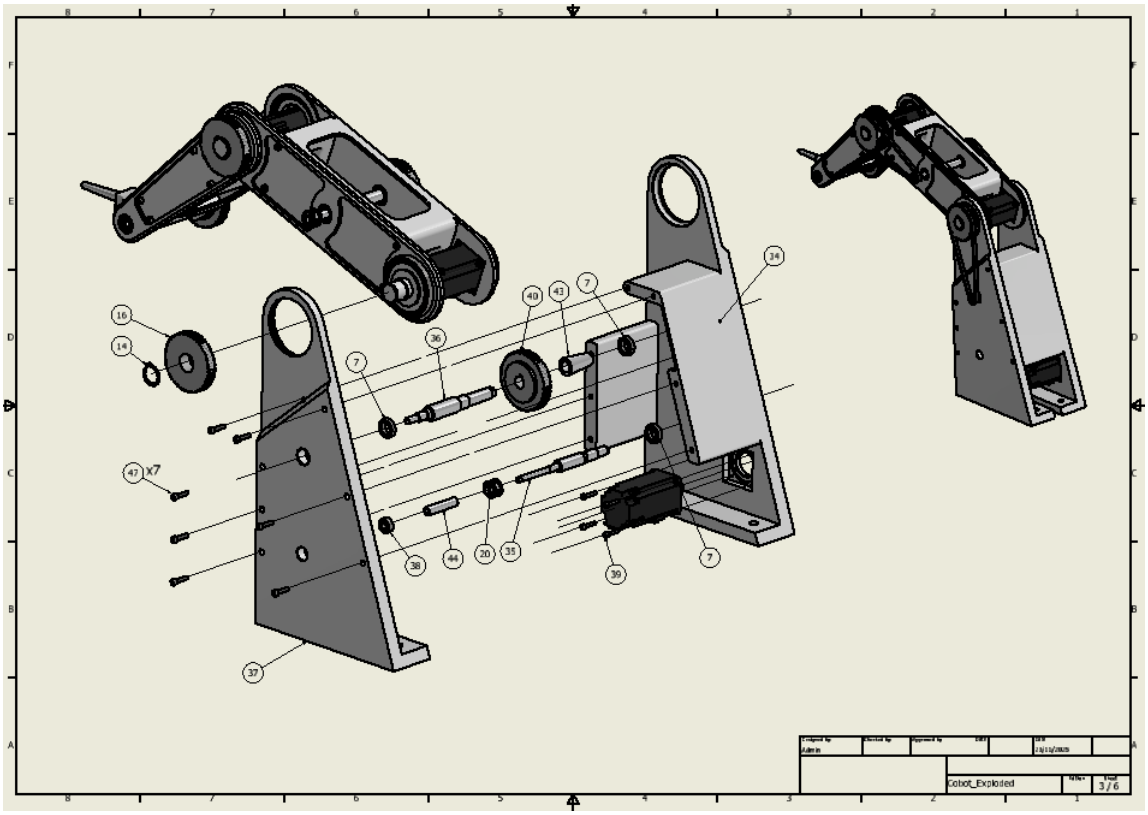
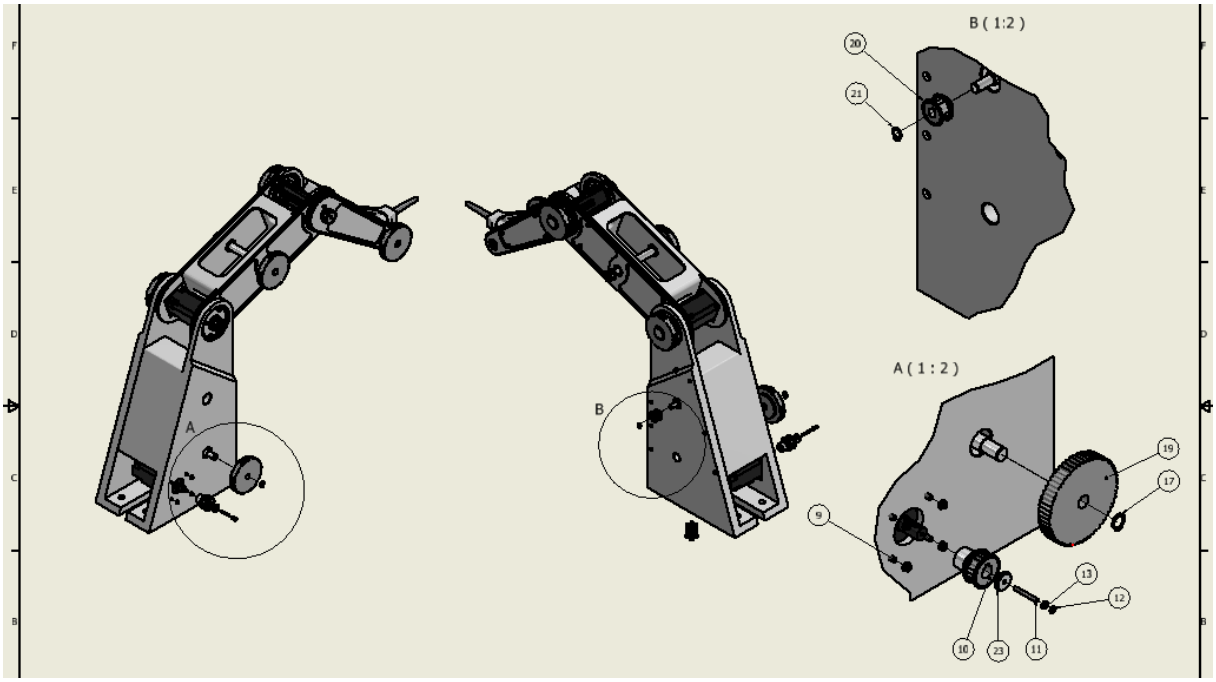


Figure 35 Assembly first arm



Assembling the base:

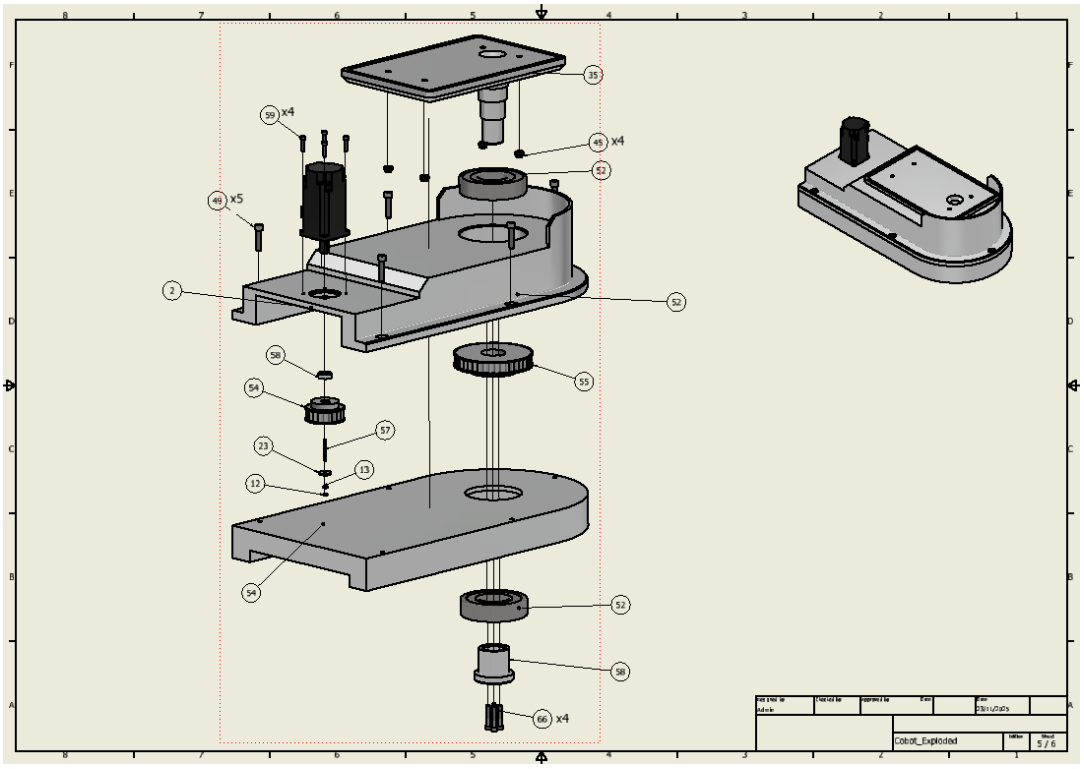


Figure 36 Assembly base

At the end we can assemble the base and the arms together:

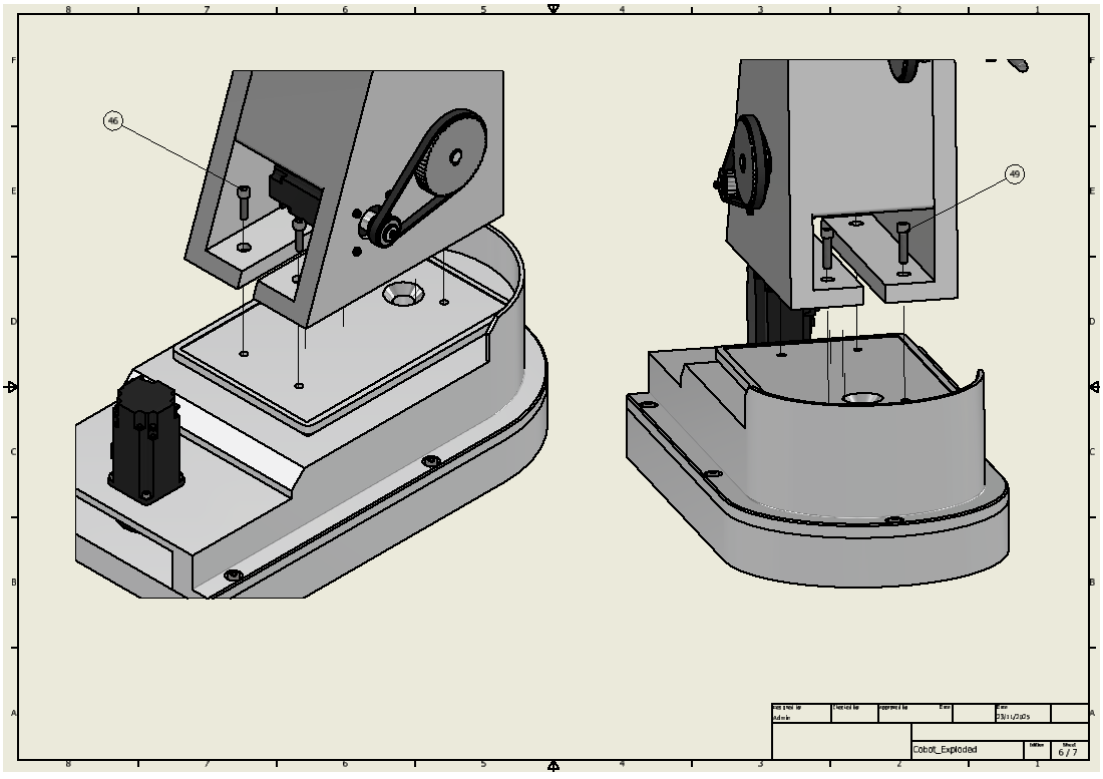


Figure 37 Assembly the arms to the base

7.2. Bearing Configurations

In our robot, we used two main bearing configurations.

The shaft at the base (the one related to the first rotation) employs a pair of **back-to-back angular-contact ball bearings**, preloaded to eliminate internal clearance and maximize radial and axial stiffness. We choose this configuration in this case because this part must sustain different type of load (both axial and radial).

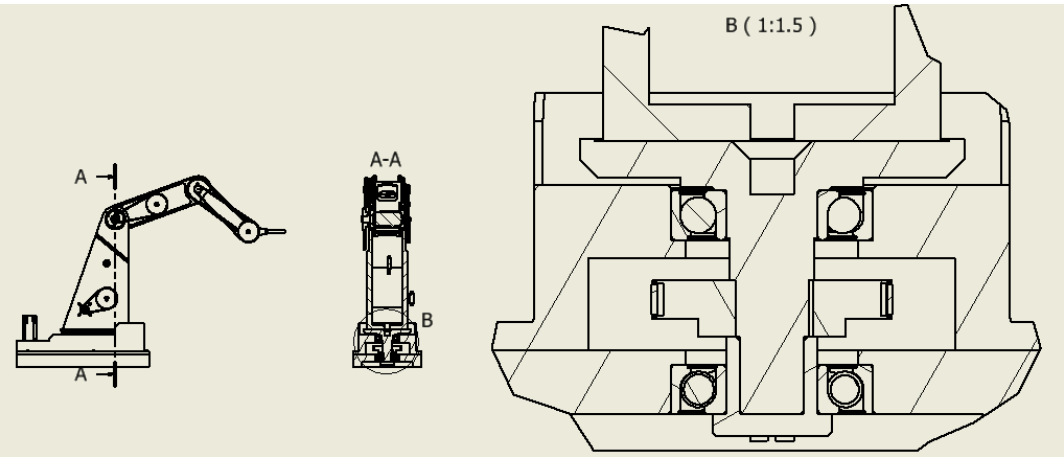


Figure 38 First bearing configuration

All the other shafts and joints use pairs of **deep-groove radial ball bearings**. Although angular-contact bearings would be preferred, in these locations axial loads are negligible (radial forces are the predominant ones). The bearings are fixed by the surrounding structure and shaft shoulders. Basically, we choose this configuration for simplicity.

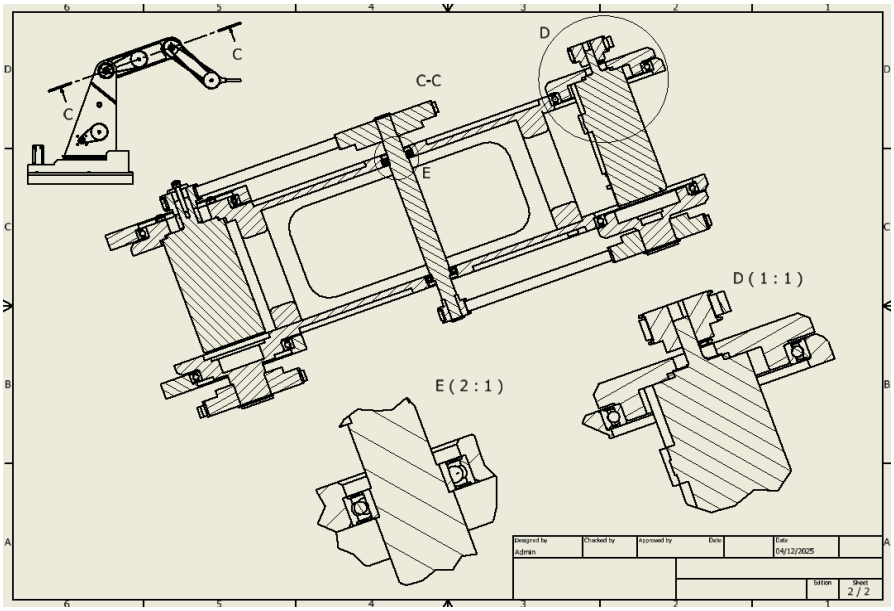


Figure 39 Second bearings configuration

7.3. Pulleys

Initially, we considered using keys (“linguette, non le chiavette”) to fix the pulleys, as they have no play (there is a slight interference) on the lateral walls, minimizing backlash and improving rotational accuracy. However, keys perform poorly under frequent reversal of rotation and can increase backlash. Therefore, for applications requiring high repeatability and many on/off cycles, **clamped, friction, or conical-bushing mounts are preferred**, as they ensure zero-backlash, even load distribution and maintain precision over time. Additionally, axial retention should be ensured using for example shoulders or circlips.

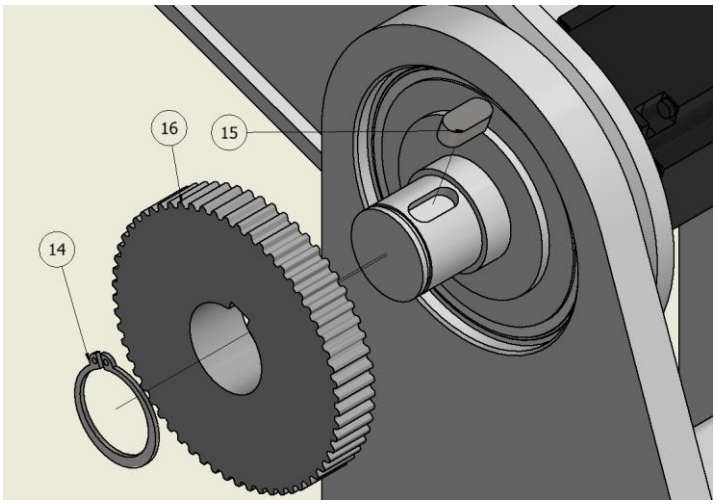


Figure 40 First idea of pulleys mounting configuration

7.4. Transmission ratios

- **Joint 1**
  The first motor rotates the first arm (and consequently all the other arm) respect to the base. We use a single transmission of 1:2.

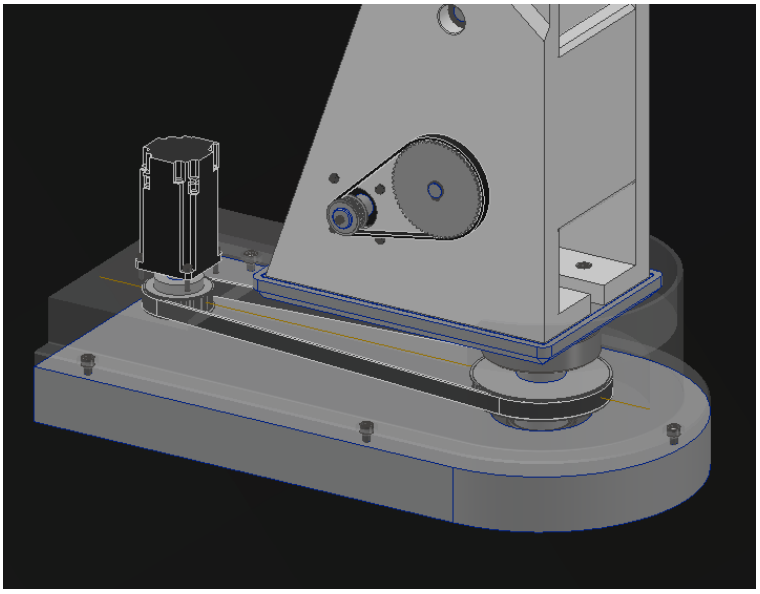


Figure 41 Trasmission ratio of tau=0.5;

- **Joint 2**
  The second motor is fixed at the left side of the first arm. This motor rotate with the first arm. This motor rotates the second arm. We use three lines of transmission the first 1:3, the second 1:4 and the third 1:4. The total transmission rate 1:48.

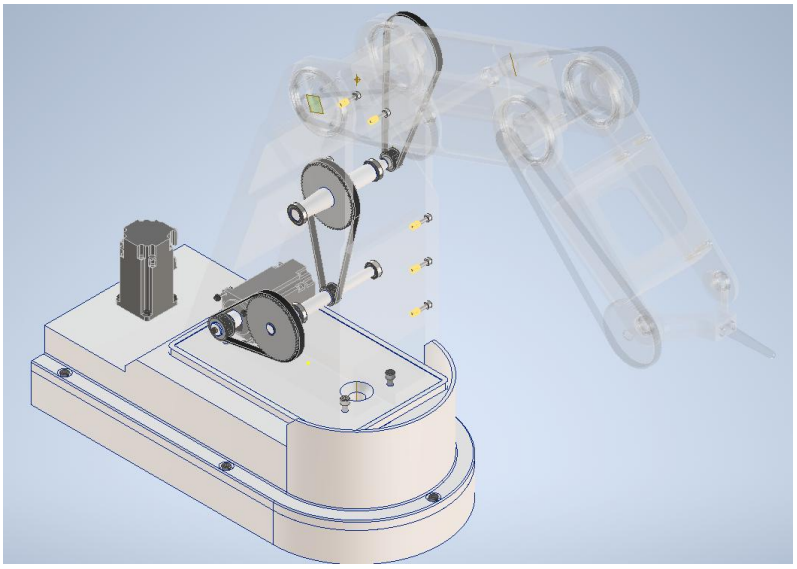


Figure 42Trasmission ratio of tau=0.02

- **Joint 3**
  The third motor is fixed at the second arm (in the left side) in the second joint. This motor rotates with the second arm. With this motor we rotate the third arm. We use two lines of transmission the first 1:3 and the second 1:3. The total transmission rate 1:9.

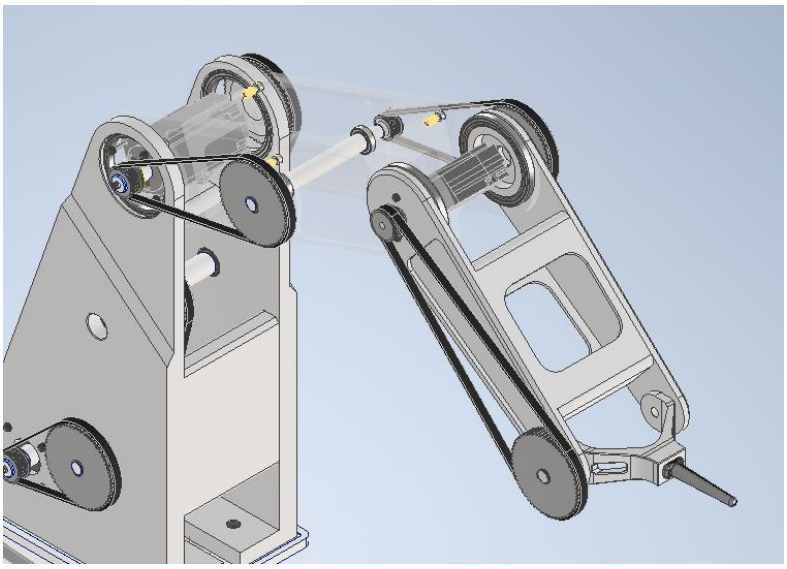


Figure 43Trasmission ratio of tau=0.11

- **Joint 4**

The fourth motor is fixed at the third arm (in the left side) in the third joint. This motor rotates with the third arm. With this motor we rotate the end effector. We use a single transmission of 1:3.

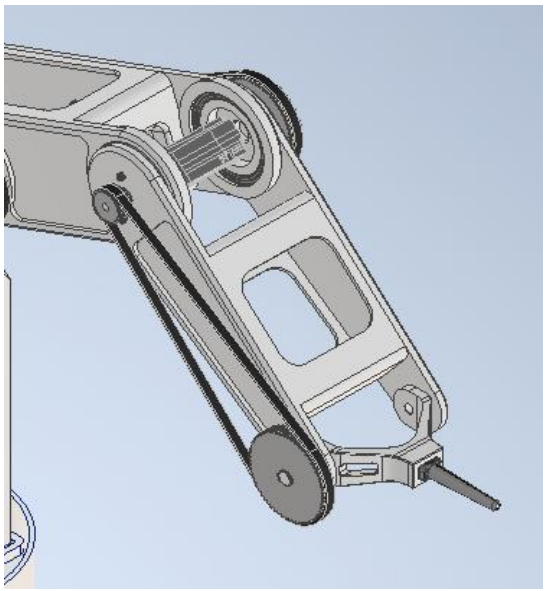


Figure 44 Trasmission ratio of tau=0.33

7.5. Solution to avoid thread ABS

To avoid threading ABS, we use Threaded Heat Insert for plastic. Threading directly into plastic is not recommended, as the thread is prone to rapid wear and deformation. A plastic thread cannot ensure stable preload over time and quickly loses its ability to create a solid, durable joint. Consequently, the mechanical reliability of the connection is significantly reduced.

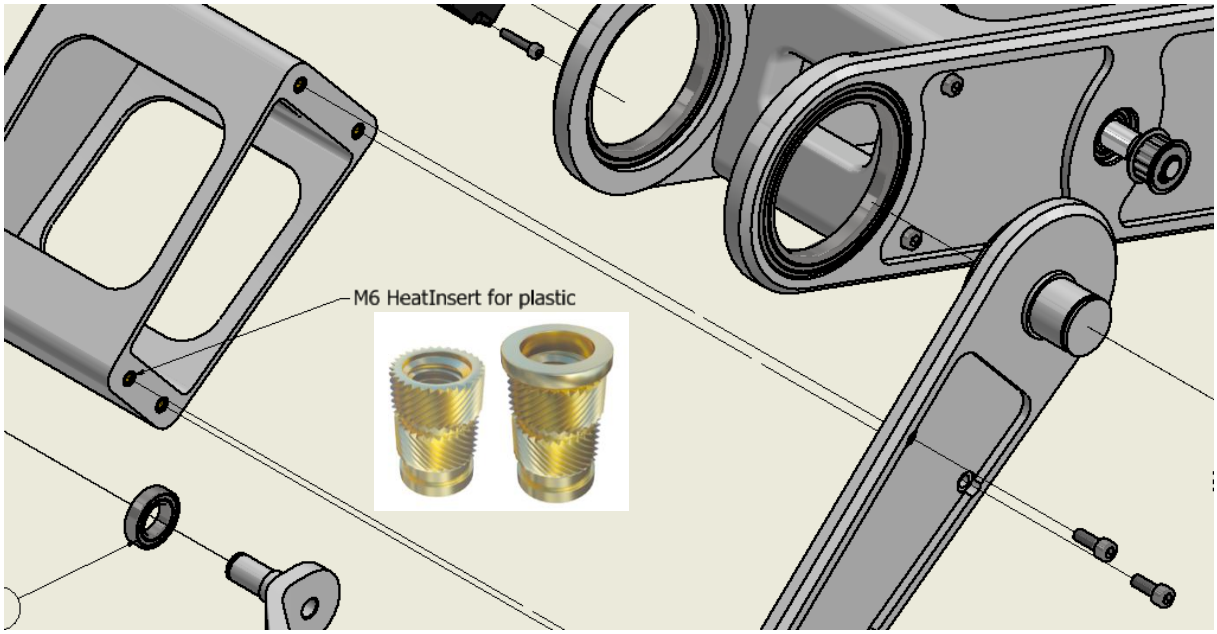


Figure 45 Heat-Insert for plastic

8 EPSON RC+ 8.0 virtual simulation

In order to simulate the robot in a virtual environment, Epson RC+ 8.0 is used, which is a software for robot development, programming and simulation. The programming language is *SPEL*, which is a high-level motion control language specially designed for Epson industrial robots. The structure of the *SPEL* language is similar to the *BASIC* language, but additionally it contains built-in functions like real-time robot motion commands, sensor and I/O interfaces, interpolation control and many others.

For the simulation tasks, the robot that is used is the Epson C4-A901S:

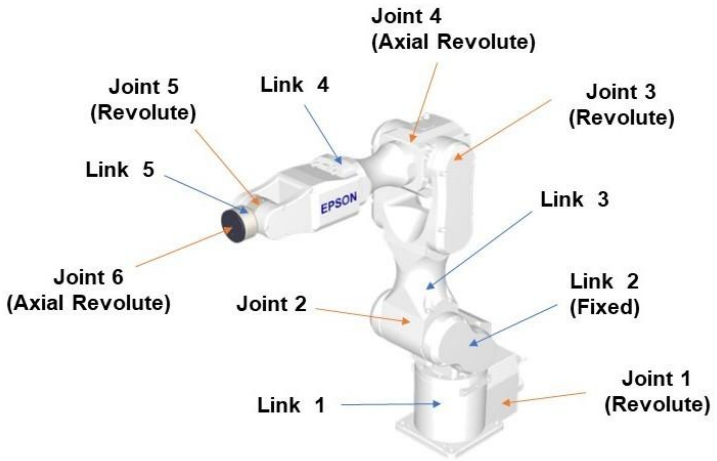


Figure 46 Epson 6-axis C4-A901S

The robot has six joints which gives the robot the 6-Axis configuration, and each of those joints represent different rotations of the links. Other important characteristics include the 900 mm maximum reach and the maximum allowable payload of 4 kg.

8.1. Robot programming

To simulate the trajectories previously defined for the 4DOF robot, a series of discretized points for both trajectories were fed into the Epson robot during "Teaching" phase. Consequently, all the motion commands were coded on the internal script.

Additionally, a gripper of length 100 mm, denominated as *Tool 1*, was assumed with the aim of simulating realistic conditions.

The following table reports the points that were used for the interpolation of the trajectories:

Teaching Points		
Geometry	**Trajectory 1**	**Trajectory 2**
Linear	P0, P1, P2, P3, P4, P5	P12, P13, P14, P15, P16, P17
Circular	P6, P7, P8, P9, P10, P11	P18, P19, P20, P21, P22, P23, P24, P25

For both the trajectories, the velocity and acceleration/deceleration (assuming that acceleration and deceleration have the same value) were set according to the average values estimated from the 4DOF robot's motion planning.

In each joint, the mean joint position was calculated:

$$\overline{q_i} = \frac{1}{N}\sum_{k=1}^{N} q_k \qquad i = 1,2,3,4$$

The joint positions were numerically differentiated and then the average speed over the trajectory was calculated using the function *trapz* in Matlab, for each joint:

$$\overline{\dot{q}_i} = \frac{1}{t_f - t_0}\int_{t_0}^{t_f} |\dot{q}|dt \; = \frac{1}{T}\int_{t_0}^{t_f} |\dot{q}|dt \qquad i = 1,2,3,4$$

With the similar procedure to the average speed, the average acceleration (magnitude) was calculated for each joint:

$$\overline{\ddot{q}_i} = \frac{1}{t_f - t_0}\int_{t_0}^{t_f} |\ddot{q}|dt \; = \frac{1}{T}\int_{t_0}^{t_f} |\ddot{q}|dt \qquad i = 1,2,3,4$$

Using the average quantities calculated previously, the "average" Jacobian and its derivative was calculated and consequently the average speed vector of the TCP was obtained:

$$\overline{\dot{S}} = J_m \overline{Q}$$

The average speed of the TCP was obtained by calculating the norm:

$$\left\|\overline{\dot{S}}\right\|_2 = \sqrt{\overline{\dot{S}_x}^2 + \overline{\dot{S}_y}^2 + \overline{\dot{S}_z}^2}$$

Finally, the average acceleration (magnitude) vector of the TCP was obtained:

$$\ddot{\bar{S}} = \dot{J}_m \dot{\bar{Q}} + J_m \ddot{\bar{Q}}$$

The average acceleration of the TCP was calculated:

$$\left\| \ddot{\bar{S}} \right\|_2 = \sqrt{\ddot{\bar{S}}_x^2 + \ddot{\bar{S}}_y^2 + \ddot{\bar{S}}_z^2}$$

After the preliminary procedure discussed above, a set of commands necessary for the robot's motion execution were written onto the Epson's scripts. The codes for the Trajectories (1 and 2) are reported below:

```
Function trajectory1()

    ' --- TRAJECTORY 1 (A-B-C) ---

    ' (1) Turn on the motors
    If Motor = Off Then
        Motor On
    EndIf

    ' (2) Set Tool from Tool 0 to Tool 1
    Tool 1

    ' (3) Save the logged data of the TCP and joints
    Xqt saveJointData("Trajectory_1.txt")

    ' It is assumed the the resting position is at P0

    ' (4) Set the Power to high and the Speed and Acceleration
    Power High
    SpeedS 220
    AccelS 482, 482

    ' ***Trajectory 1 - Linear (A-B)***

    ' (5) Perform the linear trajectory
    Move P 0
    Move P 5

    ' ***Trajectory 1 - Cicular (B-C)***

    ' (6) Perform the circular trajectory
    Curve "Circ1", 0, 0, 6, P 6, P 7, P 8, P 9, P 10, P 11
    CVMove "Circ1" CP

    ' *** Trajectory 1 - Reverse

    ' (6.I) Perform the circular trajectory (in reverse)
    Curve "Circ3", 0, 0, 6, P 11, P 10, P 9, P 8, P 7, P 6
    CVMove "Circ3" CP

    ' (6.II) Perform the linear trajectory (in reverse)
    Move P 5
    Move P 0

    ' (7) Decelerate and turn off the motors
    Motor Off

    ' (8) Quit logging the data
    Quit saveJointData

    Fend
```

Figure 47 Trajectory 1 motion code

```
Function trajectory2()

' --- TRAJECTORY 2 (A-D-E) ---

' (1) Turn on the motors
If Motor = Off Then
    Motor On
EndIf

' (2) Set Tool from Tool 0 to Tool 1
Tool 1

' (3) Save the logged data of the TCP and the joints
Xqt saveJointData("Trajectory_2.txt")

' It is assumed the the resting position is at P12 (equal to P0)

' (4) Set the Power to high and the Speed and Acceleration
Power High
SpeedS 314
AccelS 546, 546

' ***Trajectory 2 - Linear (A-D)***

' (5) Perform the linear trajectory
Move P12
Move P17

' ***Trajectory 2 - Circular (D-E)***

' (6) Perform the circular Trajectory
Curve "Circ2", 0, 0, 6, P18, P19, P20, P21, P22, P23, P24, P25
CVMove "Circ2" CP

' (7) Decelerate and turn off the motors
Motor Off

' (8) Quit logging the data
Quit saveJointData

Fend
```

Figure 48 Trajectory 2 motion code

```
Function saveJointData(FileName$ As String)

    ' Function to save into a txt file, the Elapsed time, TCP Position and Speed
, and the Joint positions
    ResetElapsedTime
    P30 = XY(0, 0, 0, 0)
    WOpen FileName$ As #30

    Do
        P30 = CurPos
        Print #30, ElapsedTime, ",", CX(P30), ",", CY(P30), ",", CZ(P30)
, ",", TCPSpeed, ",", Agl(1), ",", Agl(2), ",", Agl(3), ",", Agl(4), ",",
Agl(5), ",", Agl(6)
        Wait 0.01

    Loop
```

Figure 49 : saveJointData function – user-defined function that logs and saves TCP and joints data

In both the trajectories, the codes above are structured based on the following algorithm:

1. **Motor state switch:** Turn on the motors in case they're turned off.
2. **Tool change:** Change the Tool from *Tool 0* (default Tool) to *Tool 1* (with assumed length of 100 mm).
3. **Initialize data acquisition:** Execute the user-defined *saveJointData* function to log data onto a .txt file, related to the position and speed of the Tool Centre Point (TCP), and the joint positions.

4. **Power Mode switch and Velocity and Acceleration setting**: Set the Power Mode to High in order for the robot to run at full speed with the full servo stiffness. Consequently, set the speed and the acceleration for the continuous motion path. The *SpeedS* function sets the speed (in mm/s) of the TCP while the *AccelS* sets the acceleration (in mm/s^2) and deceleration rates of the TCP. The speed and acceleration values for both the trajectories were set based on the numerical estimate of the average values in the 4DOF robot being studied.
5. **Linear trajectory execution:** Perform the linear trajectory using the function *Move*, from the initial point $P_{initial}$ (also assumed as the resting point) to the final point P_{final}. During the linear motion, the intermediate points between the initial and final point will automatically be interpolated linearly, therefore only two points were provided. In trajectory 1 the reverse motion is performed.
6. **Circular trajectory execution:** Perform the circular trajectory using the function *Curve* and *CMove*. The function Curve receives six input parameters: the name of the *.cvt* file in which point data will be saved as a string, the specification whether the curve to be performed is closed ("C") or open ("O"), the mode setting whether or not the arm is automatically interpolated in the tangential direction of some axis, the number of axes (which is 6, for the current case), and the list of points to be interpolated. Finally, the *CVMove* performs the continuous spline path motion defined by the *Curve* function. In trajectory 1 the circular path is performed in reverse.
7. **Deceleration and Motor state switch:** Turn off the motors using the *Motor Off* command. Since previously continuous path motion functions are used, such as the function Curve which needs a continuous functioning of the motors, the *Motor Off* function permits the robot motors to decelerate until the motors are turned off.
8. **Terminate data acquisition:** Stop the execution of the *saveJointData* to quit the acquisition of the data from the TCP and joints.

The codes presented above are structurally similar, the only main difference lies in the fact that the trajectory 1 also includes executing the same geometric path in reverse.

8.2. Trajectory motion simulation

After the completion of the code development, both the trajectories were simulated on the Epson's internal robot simulator. Inside the simulator, the EPSON robot was placed in a controlled area, and additionally the inertial frames regarding both the robot and the virtual gripper are visible. In order to enrich the simulation, the similar helmet presented previously was placed inside the simulation area. The helmet was assumed to have an offset of 300 mm with respect to the x-y plane.

After running the trajectory code, the robot was set in motion according to the programmed commands and the virtual gripper (Tool 1) consequently traced the motion path (depicted in yellow).

8.2.1. Trajectory 1 simulation

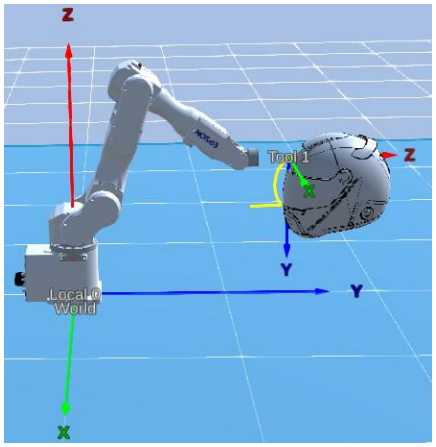


Figure 50 Simulation of the Trajectory 1

As expected for the simulation, the Trajectory 1 traced a planar curve on the z-y plane. The linear and the circular path are well interpolated. The entire trajectory was completed in 5 seconds.

8.2.2. Trajectory 2 simulation

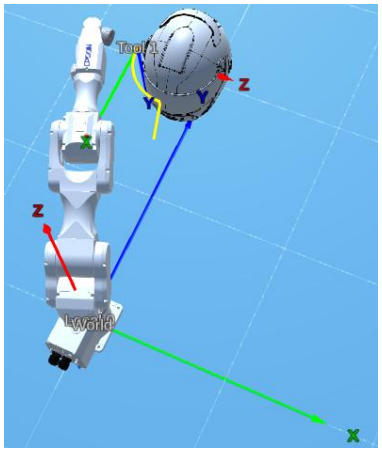


Figure 51 Simulation of the Trajectory 2

It can be observed that trajectory 2 traced a satisfactory interpolation of the linear and circular paths. For simplicity, the last link, which is connected to the 6° joint, is

rotated 90° clockwise, in order to satisfy the geometry of the motion path (however, other configurations may exist). The trajectory was completed in 5 seconds.

8.3. TCP: Position, Velocity and Acceleration

Within the timeframe of the simulation of the trajectories, the data regarding the position, velocity and acceleration of the TCP and the joints were acquired with a particular sampling frequency and saved in a file. The acquired data were then imported and plotted in Matlab. In order to improve the smoothness of some of the plots, a filtering procedure was implemented, using the **Savitzky-Golay filter**.

8.3.1. Position – Trajectory 1 and Trajectory 2

The TCP position plots of the Trajectory 1 are represented below:

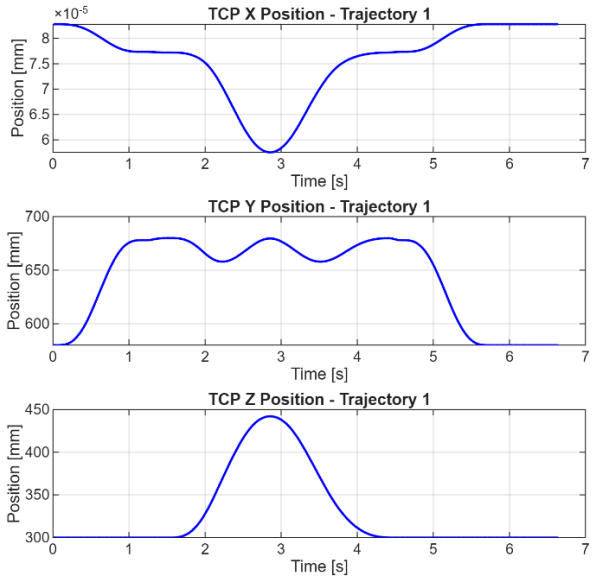


Figure 52 The plots represent the TCP position of the Trajectory 1 respectively on the x, y and z axis

It can be noticed that the TCP position of the x coordinate, which in reality should be zero (since the Trajectory 1 is planar and belongs to the z-y plane), give extremely low values due to the numerical noise arising from the discretized simulation time, floating-point precision, and the virtual servo quantization.

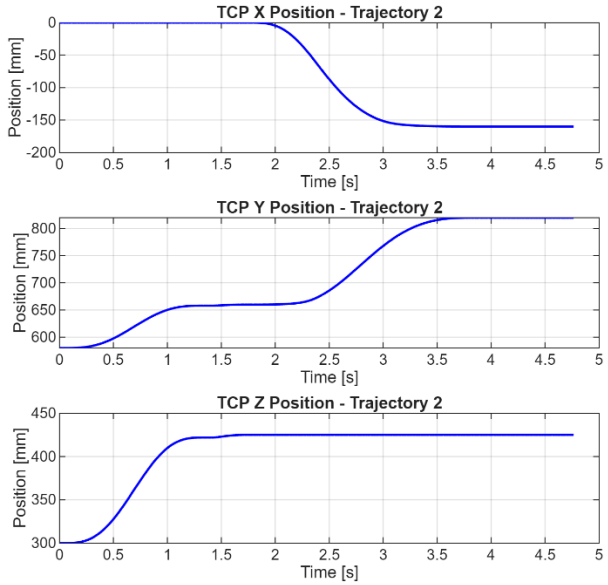


Figure 53 : The plots represent the TCP position of the Trajectory 2 respectively on the x, y and z axis

Reconstructing the trajectory points simulated earlier, the TCP motion path traced by the robot for both the trajectories are reported below:

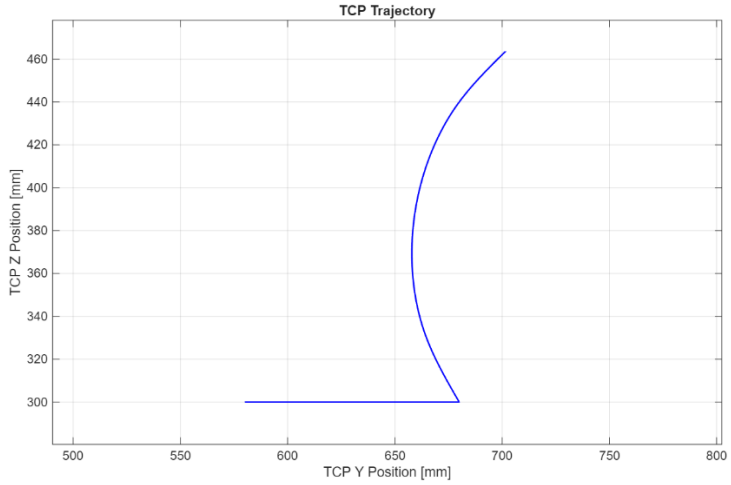


Figure 54 Trajectory 1 curve

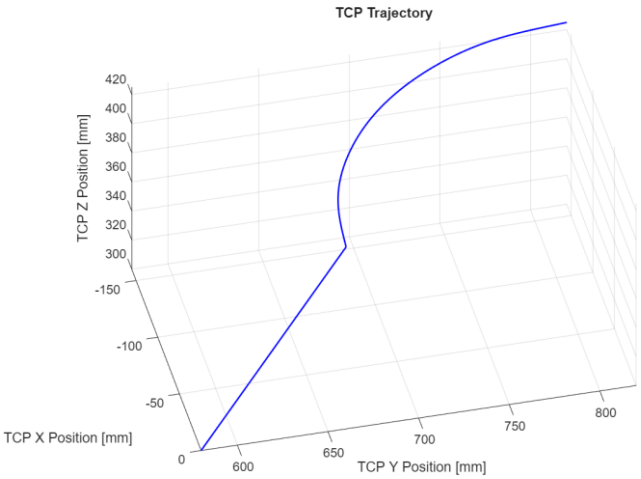


Figure 55 Trajectory 2 curve

Similar to the simulated trajectories from Epson's simulator, the reconstructed trajectory plots, using Matlab, show smooth and well reproduced motion paths.

8.3.2. Speed – Trajectory 1 and Trajectory 2

The TCP velocities of each axis were obtained differentiating numerically using the *gradient* function of Matlab, the TCP position of each coordinate. Some irregularities in the plot are present since differentiation amplifies the numerical noise already present in the data. On the other hand, the TCP speed of each trajectory was obtained using the data logger function described earlier. It can be noticed that by calculating the Euclidean norm of the TCP velocity (since the velocity has x, y and z components), it overlaps the TCP speed plot, with a few deviations due to the amplified noises during the derivation process.

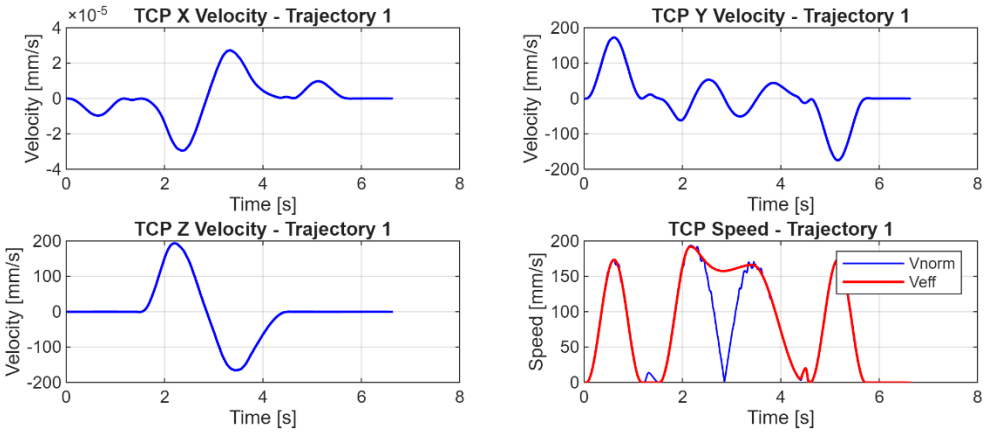


Figure 56 TCP Velocities of each component and TCP Speed plots of the Trajectory 1

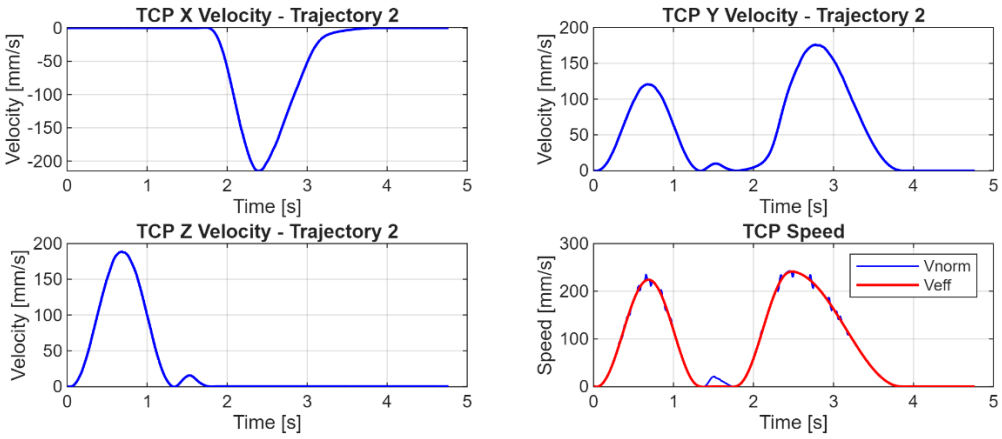


Figure 57 TCP Velocities of each component and TCP Speed plots of the Trajectory 2

On the TCP Speed plots, V_{norm} (depicted in blue) represent the norm of the TCP velocity, while the V_{eff} (depicted in red) represents the TCP speed acquired from the data logger. In trajectory 1, a significant difference between the plots of V_{norm} and V_{eff} can be observed, which happens in the inversion point or when the robot executes the motion in reverse. The in the inversion point (when the TCP starts returning to the origin) a possible numerical error occurs due to the numerical differentiation. Some plausible explanations could be that the sharp drop of the V_{norm} plot was a direct cause of numerical differentiation, or that during the inversion, the robot theoretically reaches zero velocity for a fraction of second, but in reality it isn't quite feasible. The V_{eff} plot seems to explain more realistically the situation since it doesn't reach zero velocity and that the reduction of speed during the transition is much smoother.

8.3.3. Acceleration -Trajectory 1 and Trajectory 2

The TCP acceleration plot for both the trajectories was obtained differentiating numerically the TCP speed yielded by the data logger function. The TCP acceleration for each axis, which would be obtained by differentiating the x, y, and z components of the TCP velocity were not reported due to the notable irregularities derived from the numerical differentiation.

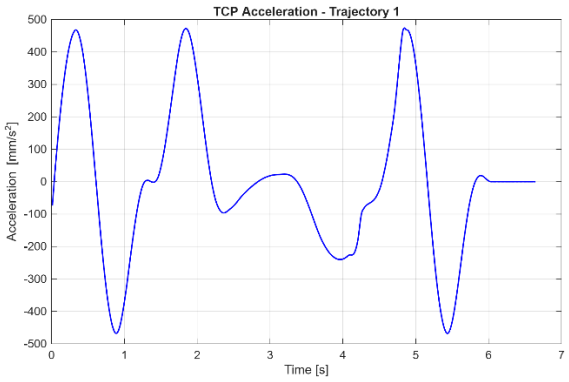


Figure 58 TCP Acceleration plot of the Trajectory 1

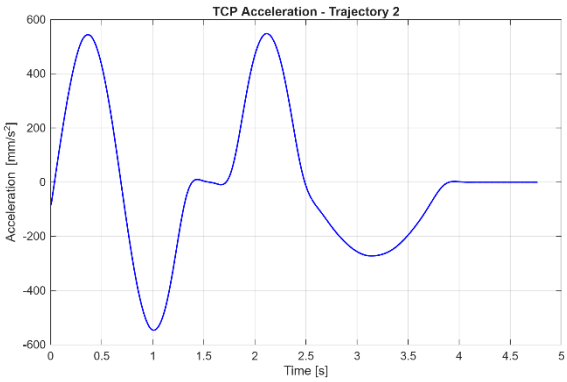


Figure 59 TCP Acceleration plot of the Trajectory

In trajectory 1, the two positive peaks represent the forward execution of the motion and then additionally the successive trends are due to the reverse motion. In trajectory 2, the peaks are also due to the forward motion, likely due to the linear and circular path. The negative peaks represent the deceleration, which occurs during the transition from the linear to the curved path and finally when the robot is near the end of the trajectory.

8.4. Joint: Position, Velocity and Acceleration

With the same procedure as previously discussed, the joint position data were acquired in parallel to the data acquisition from the TCP. In Epson however, there are no available functions to acquire the joint velocity and acceleration and the only way to obtain them is to do a numerical differentiation through Matlab. As stated previously, problems due to numerical noises after differentiation also persist and to smooth out the irregularities a filtering procedure using Savitzky-Golay filter was also implemented.

8.4.1. Joint Position – Trajectory 1 and Trajectory 2

The joint positions acquired through the data logger were reported below:

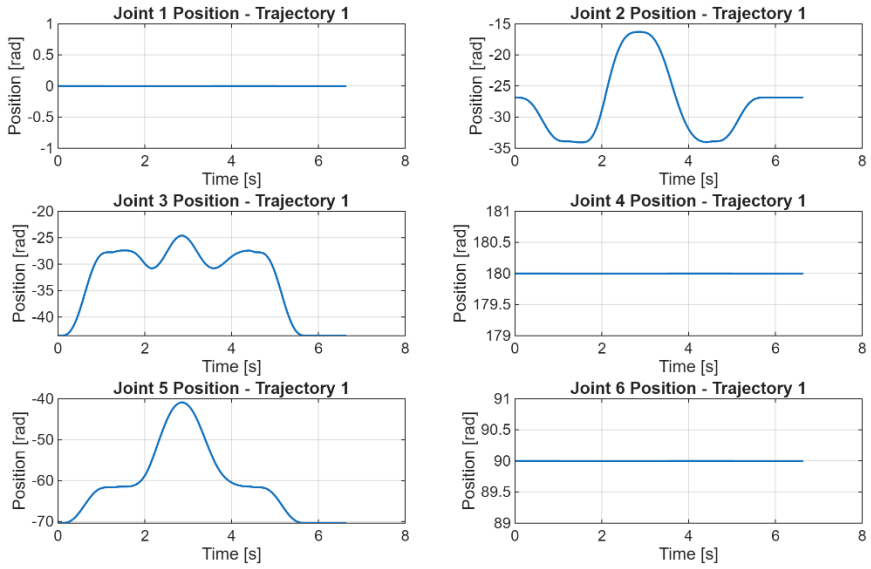


Figure 60 Joint position plots of Trajectory 1

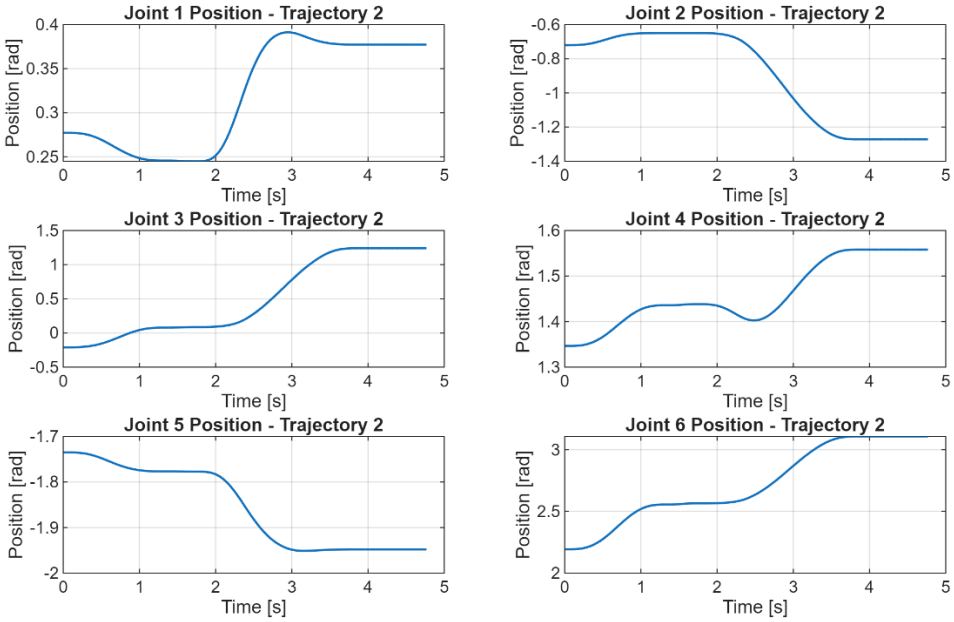


Figure 61 Joint position plots of Trajectory 2

The plots above for both the trajectories show a smooth trend such that the filtering was omitted. In the Trajectory 1 a symmetry in the plots could be observed due to the forward and reverse motion , and it can also be seen that the joints 1, 4 and 6 are at rest since the motion is planar and only the joints 2, 3 and 5 are involved. In Trajectory 2,

all the plots are all consistent to the motion path since it can be observed that all the joints are involved.

8.4.2. Joint Velocity – Trajectory 1 and Trajectory 2

The joint velocities were obtained differentiating numerically the joint positions using the *gradient* function in Matlab. Below, the joint velocity plots are reported:

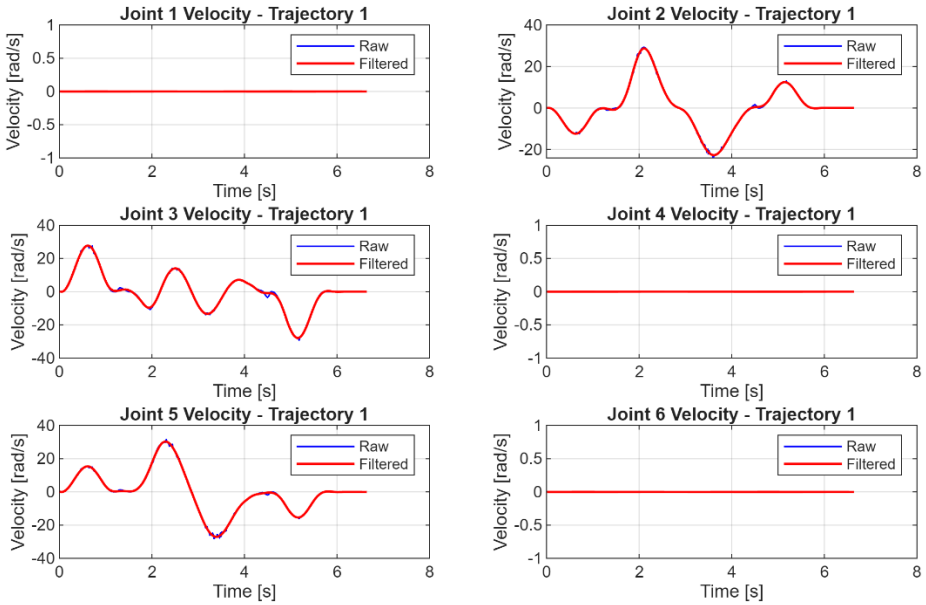


Figure 63 Joint velocity plots of Trajectory

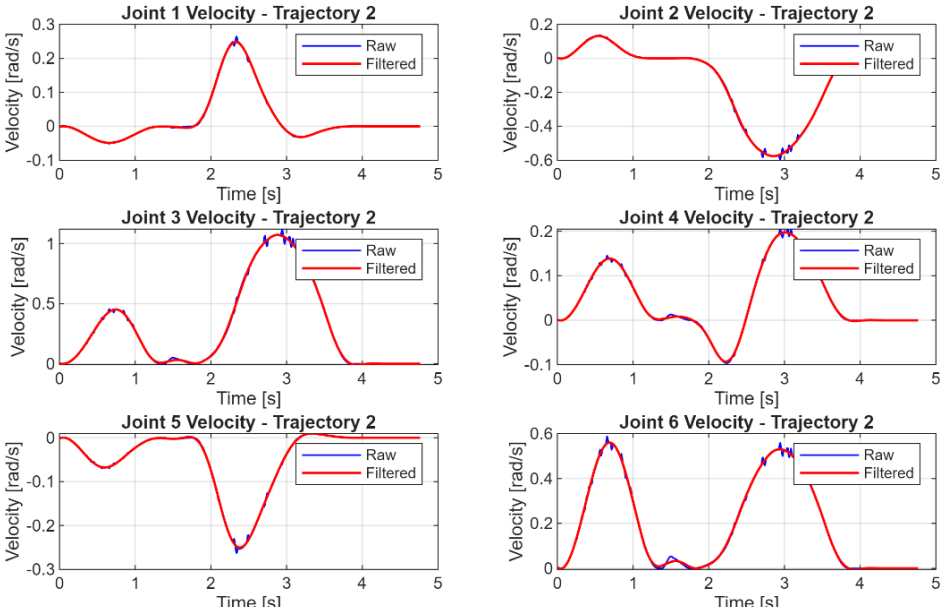


Figure 62 Joint velocity plots of Trajectory 2

It can be observed that for both the trajectories, their unfiltered joint velocity plots are slightly noisy with respect to the filtered version. The plots of both the trajectories are likewise consistent with the joint motion.

8.4.3. Joint Acceleration – Trajectory 1 and Trajectory 2

Analogously to the joint velocities, the joint accelerations were obtained through the numerical differentiation of the joint velocities. The joint acceleration plots are reported below:

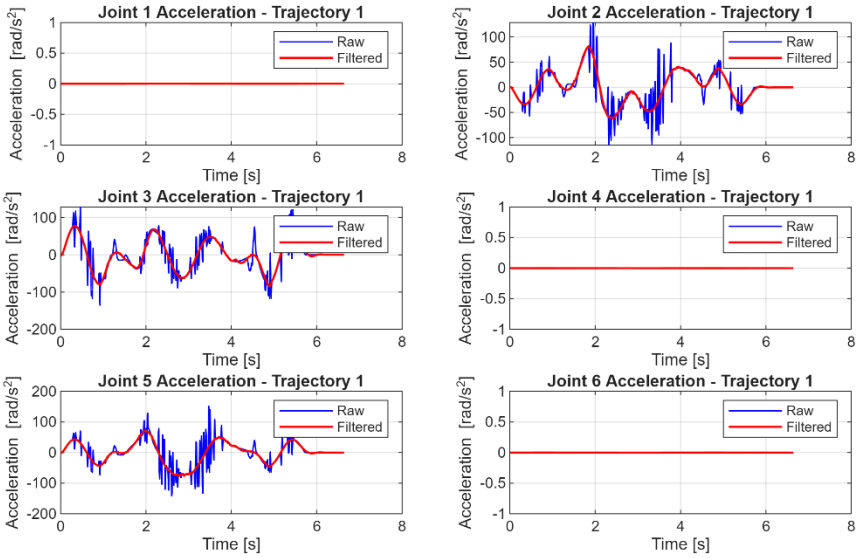


Figure 64 Joint acceleration plots of Trajectory 1

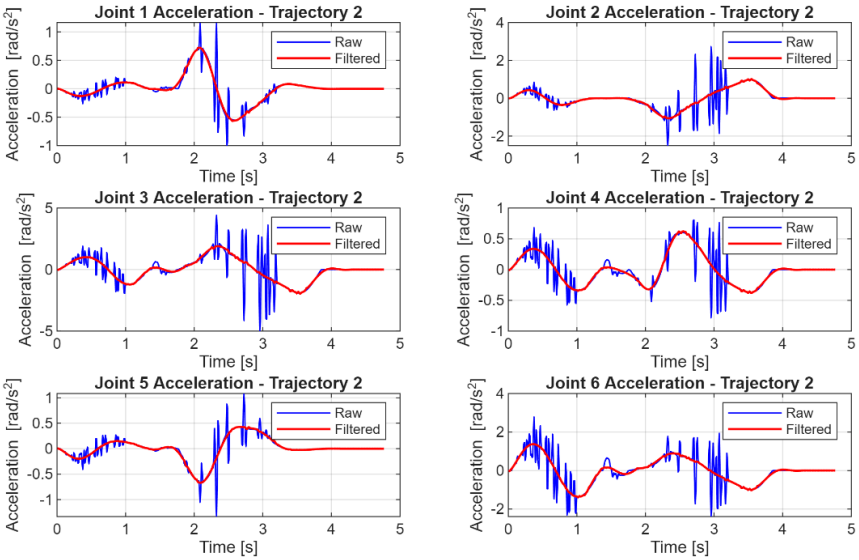


Figure 65 Joint acceleration plots of Trajectory 2

As can be observed above, significant noises on the unfiltered plots of the joint accelerations are present due to the noise amplification caused by the differentiation procedure. With proper filtering, the noise spikes were attenuated.